

MFJA 3rd Floor Gazebo Harmonic
Current Multi-Robot and Room 315 Kinematic
Shuttle Report

Technical Project Report

April 20, 2026

Contents

1	Purpose and Current Scope	4
2	Repository Architecture	6
2.1	Meta-repository layout	6
2.2	Build command	6
3	Multi-Robot Simulation Layer	9
3.1	Supported robots	9
3.2	Robot registries	11
3.3	Full floor launch	11
3.4	Room 315 only launch	11
3.5	Robot topic checks	11
3.6	Example one-robot commands	11
3.7	Synchronized multi-robot tool	12
4	Room 315 Kinematic Shuttle Layer	13
4.1	Rail network model	13
4.2	Current start slots	13
4.3	Starting the shuttle node	13
4.4	Switch primitive	14
4.5	Stopper primitive	14
4.6	Sensor workflow	17
4.7	Manual stop-switch-release workflow	18
4.8	Per-shuttle ON/OFF control	18
4.9	Runtime add behavior	18
4.10	Collision avoidance	19
5	Validation Procedures	21
5.1	CSV and graph validation	21
5.2	Runtime topics to inspect	21
5.3	Expected negative tests	22
6	Current Operational Notes	24
7	Detailed Implementation Walkthrough	25
7.1	Design rule: topology is explicit, geometry is only geometry	25
7.2	How the rail paths were built from CSV files	25
7.3	Start slots and physical entry poses	33
7.4	ROS node parameter model	35

7.5	Loading start slots from the network file	37
7.6	Stopper configuration and stop-point generation	37
7.7	Switches, fixed transitions, and routing table	39
7.8	Pure Python kinematic core	41
7.9	Arc-length path backends	42
7.10	Continuous path validation	44
7.11	Successor resolution and explicit failure	46
7.12	Core stepping logic	47
7.13	Runtime shuttle creation	47
7.14	Stopper command implementation	49
7.15	Per-shuttle ON/OFF implementation	50
7.16	Motion guard order	52
7.17	Collision avoidance behavior	54
7.18	Sensor event publication	55
7.19	Published state payload	56
8	Complete Feature Usage Manual	57
8.1	Terminal setup used by all examples	57
8.2	Complete feature checklist	57
8.3	Terminal 1: run Room 315 only	58
8.4	Terminal 1 alternative: run the full floor	58
8.5	Terminal 1 robot launch options	59
8.6	Robot terminal A: verify robot topics	60
8.7	Robot terminal B: command individual robot arms	60
8.8	Robot terminal C: command the TIAGo base	61
8.9	Robot terminal D: synchronized multi-robot commands	61
8.10	Terminal 2: bridge the Gazebo set-pose service	62
8.11	Terminal 3: run one shuttle	62
8.12	Terminal 3: four-shuttle startup	63
8.13	Terminal 4: inspect state continuously	63
8.14	Terminal 5: inspect sensor events	64
8.15	Switch commands	64
8.16	Stop-switch-release workflow	64
8.17	Add shuttles during simulation	65
8.18	Turn individual shuttles ON or OFF	65
8.19	Pose calibration controls	66
8.20	Validation commands	66
8.21	Path backend selection	67
8.22	Troubleshooting table	67
9	What Was Built in This Phase	70
9.1	Repository cleanup and package separation	70
9.2	Dynamic shuttle work removed from the current main path	70
9.3	Room 315 rail graph and validation pipeline	71
9.4	Continuous path backend added	71
9.5	Switches made independent and visible	72
9.6	Stopper primitive added before switches	72
9.7	Sensor workflow added	72

9.8	Per-shuttle control added	72
9.9	Runtime shuttle addition with occupied-slot rejection	73
9.10	Simple collision avoidance added	73
9.11	Room-only and full-floor compatibility	73
9.12	Shuttle mesh and pose alignment	73
10	Developer Notes for Future Work	74
10.1	Why stoppers are not blocks yet	74
10.2	Why collision avoidance is not full traffic control	74
10.3	Recommended next implementation phase	74
11	Conclusion	75

Chapter 1

Purpose and Current Scope

This report documents the current accepted state of the `mfja_3rd_floor_gz` meta-repository. The project now combines two main layers:

- a reusable Gazebo Harmonic and ROS 2 Jazzy multi-robot simulation for the MFJA third floor;
- a kinematic-first Room 315 shuttle simulation for the flexible rail network.

The current shuttle work is intentionally kinematic. The shuttle does not use wheel contact, rail contact dynamics, or full rigid-body contact as the primary motion source. Instead, the rail path is represented as CSV geometry plus an explicit directed graph. The node computes the shuttle pose along rail segments and sends pose updates to Gazebo through `/world/<world_name>/set_pose`.

Dynamic shuttle work is not part of the current main version. The current main version focuses on:

- independent robot control through namespaced ROS topics;
- a clean ROS 2 meta-repository package structure;
- Room 315 kinematic shuttle routing;
- independent switch states;
- independent binary stoppers before switches;
- sensor-style approach events for the teaching workflow;
- per-shuttle ON/OFF commands;
- runtime shuttle spawning;
- explicit rejection when a requested start slot is occupied;
- simple center-distance collision avoidance between shuttles.

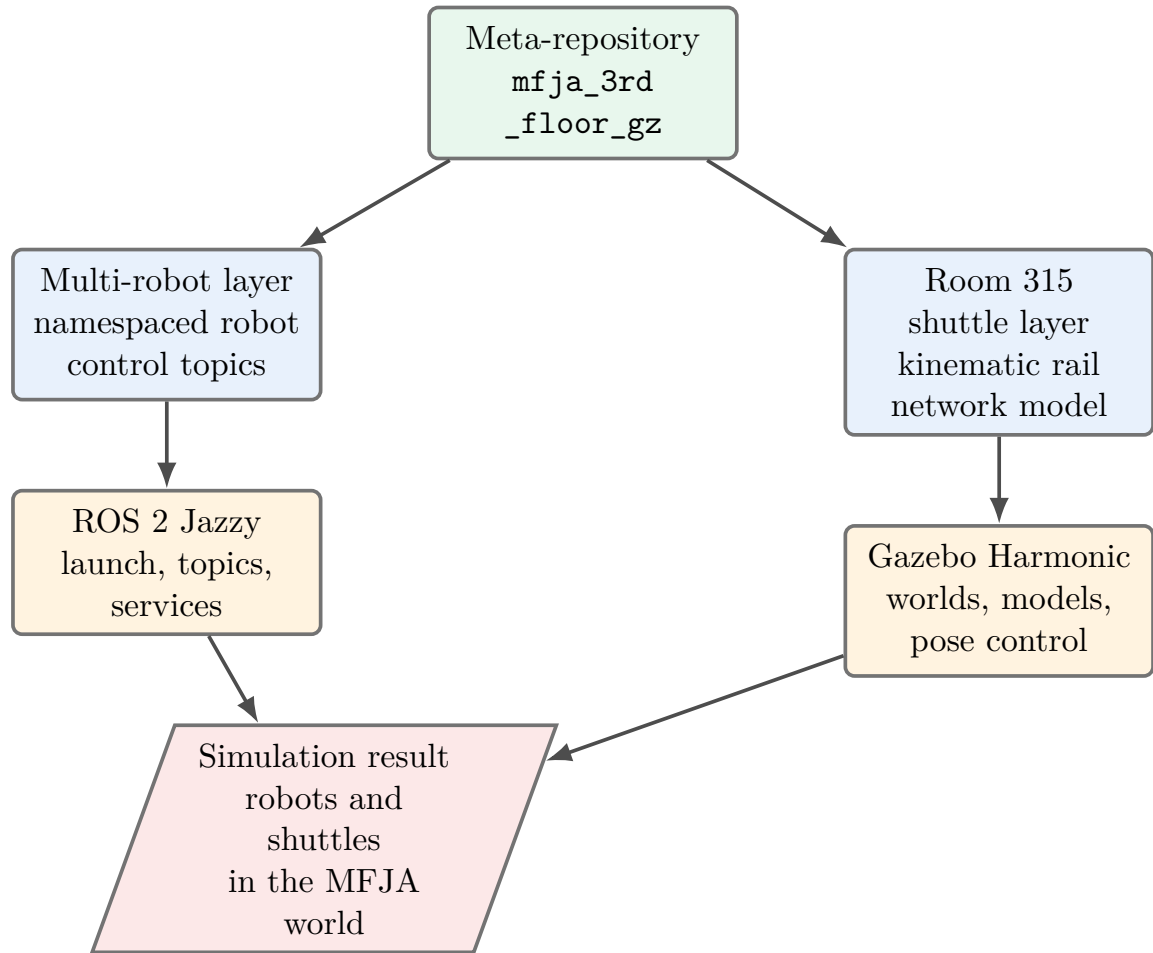


Figure 1.1: High-level project scope. The repository now contains a general multi-robot simulation layer and a Room 315 kinematic shuttle layer. Both are integrated through ROS 2 and Gazebo.

Chapter 2

Repository Architecture

2.1 Meta-repository layout

The Git repository root is intentionally not a ROS 2 package. The root contains documentation and repository-level files only. ROS packages live in subfolders:

- `mfja_3rd_floor_description`: shared Gazebo models, meshes, worlds, and URDF/SDF assets;
- `mfja_robot_control_config`: shared launch logic, bridge configuration, robot registries, Room 315 kinematic configuration, and control scripts;
- `mfja_room_315_bringup`: launch entry point for Room 315 only;
- `mfja_3rd_floor_bringup`: launch entry point for the full floor;
- `mfja_3rd_floor_gz`: umbrella compatibility package that keeps historical launch entry points available.

The root should not contain package-level files such as root `package.xml`, root `CMakeLists.txt`, root `models/`, root `worlds/`, root `launch/`, root `config/`, or root `CSV/`. Package-owned files belong inside the package that owns them.

2.2 Build command

The explicit `-base-paths` command is the recommended build method, because the repository contains several ROS 2 packages inside one Git repository.

```
cd /home/tiago/ALI_ros2_ws
source /opt/ros/jazzy/setup.bash

colcon build --base-paths \
  src/mfja_3rd_floor_gz/mfja_robot_control_config \
  src/mfja_3rd_floor_gz/mfja_3rd_floor_description \
  src/mfja_3rd_floor_gz/mfja_room_315_bringup \
  src/mfja_3rd_floor_gz/mfja_3rd_floor_bringup \
  src/mfja_3rd_floor_gz/mfja_3rd_floor_gz \
  --packages-select \
  mfja_robot_control_config \
  mfja_3rd_floor_description \
  mfja_room_315_bringup \
  mfja_3rd_floor_bringup \
  mfja_3rd_floor_gz \
  --symlink-install

source install/setup.bash
```

Listing 2.1: Current meta-repository build command.

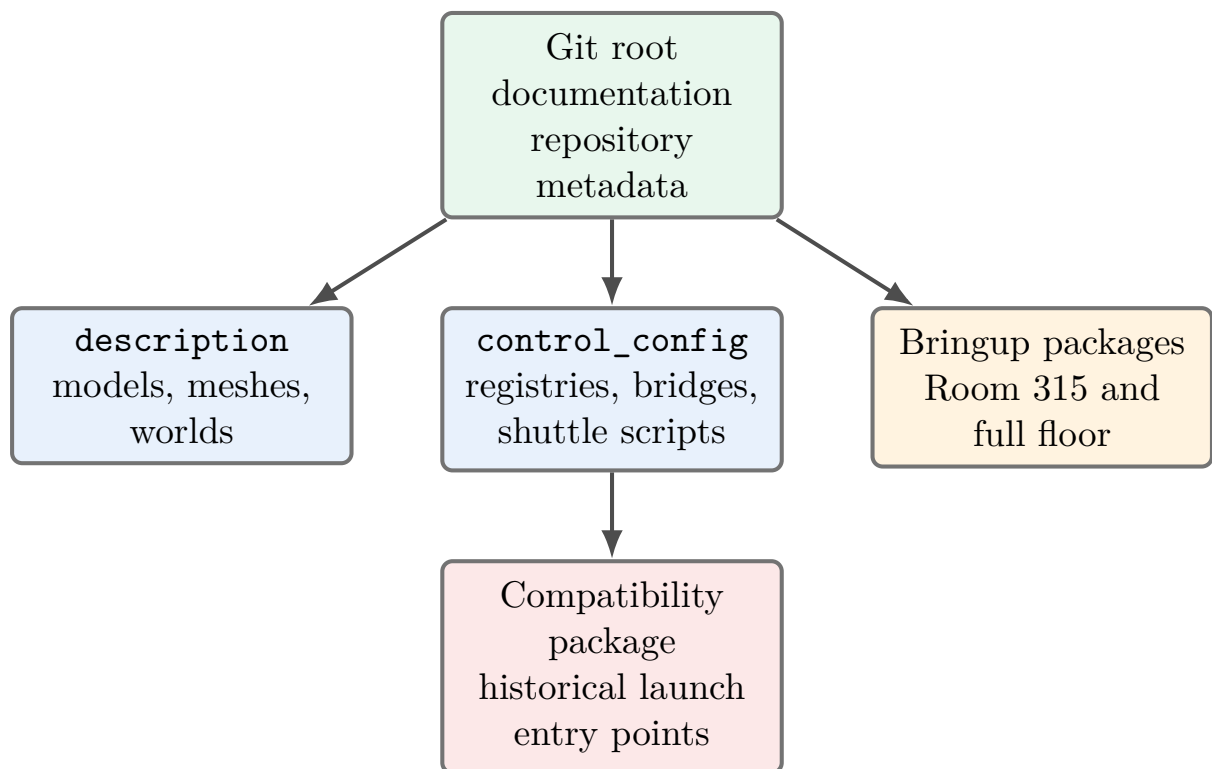


Figure 2.1: Repository ownership model. The root is not a ROS 2 package. Runtime assets live inside the package that owns them, which keeps the workspace buildable and easier to maintain.

Chapter 3

Multi-Robot Simulation Layer

3.1 Supported robots

The current simulation supports these robot instances:

Instance	Robot family		Primary command topic
kuka1	KUKA	KR6 R900 sixx	/kuka1/joint_trajectory
staubli1	Staeubli	TX2-60L	/staubli1/joint_trajectory
yaskawa_hc10_1	Yaskawa HC10		/yaskawa_hc10_1/joint_trajectory
yaskawa_hc10dt_1	Yaskawa HC10DT		/yaskawa_hc10dt_1/joint_trajectory
tiago1	TIAGo	mobile manipulator	/tiago1/joint_trajectory, /tiago1/cmd_vel

Each robot uses its own namespace. A command sent to one namespace does not affect another robot.

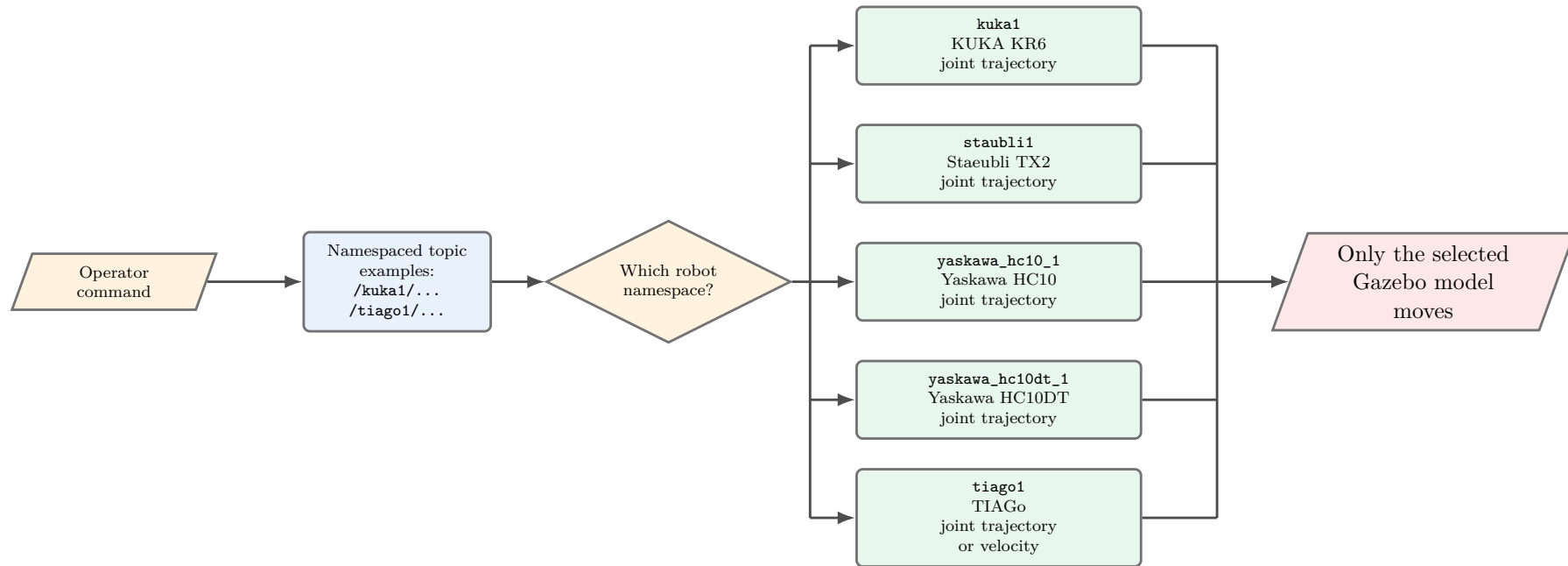


Figure 3.1: Namespaced robot control for all supported robot instances. Each command is routed through one robot namespace, so commanding one robot does not move the others.

3.2 Robot registries

Robot identity, model selection, pose, and activation are stored in YAML registries:

- `mfja_robot_control_config/config/robots.yaml` for the full floor;
- `mfja_robot_control_config/config/robots_room_315_only.yaml` for Room 315 only.

This makes robot deployment data-driven. A developer can adjust active robots and placements without editing the launch implementation.

3.3 Full floor launch

```
cd /home/tiago/ALI_ros2_ws
source /opt/ros/jazzy/setup.bash
source install/setup.bash

ros2 launch mfja_3rd_floor_bringup full_floor.launch.py \
  robots:=none \
  start_paused:=false \
  gui:=true
```

Listing 3.1: Launching the full floor.

3.4 Room 315 only launch

```
cd /home/tiago/ALI_ros2_ws
source /opt/ros/jazzy/setup.bash
source install/setup.bash

ros2 launch mfja_room_315_bringup room_315_only.launch.py \
  robots:=none \
  start_paused:=false \
  gui:=true
```

Listing 3.2: Launching Room 315 only.

3.5 Robot topic checks

```
ros2 topic list | grep -E '^(kuka1|staubli1|yaskawa_hc10_1|yaskawa_hc10dt_1|tiago1)/'

ros2 topic info /kuka1/joint_trajectory
ros2 topic info /staubli1/joint_trajectory
ros2 topic info /yaskawa_hc10_1/joint_trajectory
ros2 topic info /yaskawa_hc10dt_1/joint_trajectory
ros2 topic info /tiago1/joint_trajectory
ros2 topic info /tiago1/cmd_vel
```

Listing 3.3: Checking robot control topics.

3.6 Example one-robot commands

```
ros2 topic pub --once /kuka1/joint_trajectory \
trajectory_msgs/msg/JointTrajectory \
"{joint_names: ['joint_a1','joint_a2','joint_a3','joint_a4','joint_a5','joint_a6'], \
points: [{positions: [0.6,-1.0,1.1,0.0,0.6,0.0], \
time_from_start: {sec: 3, nanosec: 0}}]}"
```

Listing 3.4: KUKA example command.

```
ros2 topic pub --once /staubli1/joint_trajectory \
trajectory_msgs/msg/JointTrajectory \
"{joint_names: ['joint_1','joint_2','joint_3','joint_4','joint_5','joint_6'], \
points: [{positions: [0.1,0.4,-0.6,0.0,0.5,0.0], \
time_from_start: {sec: 3, nanosec: 0}}]}"
```

Listing 3.5: Stäubli example command.

```
ros2 topic pub -r 20 /tiago1/cmd_vel geometry_msgs/msg/Twist \
"{linear: {x: 0.15, y: 0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 0.20}}"
```

Listing 3.6: TIAGo base example command.

3.7 Synchronized multi-robot tool

The script `multi_robot_sync_demo.py` can publish coordinated commands from one ROS 2 process. It is better than starting several shell commands manually when multiple robots should begin a scenario together.

```
cd /home/tiago/ALI_ros2_ws
source /opt/ros/jazzy/setup.bash
source install/setup.bash

ros2 run mfja_robot_control_config multi_robot_sync_demo.py --list-joints
```

Listing 3.7: Inspecting supported synchronized joints.

```
ros2 run mfja_robot_control_config multi_robot_sync_demo.py \
--goal kuka1=1.2,-1.2,1.4,0.0,0.3,0.0 \
--goal staubli1=0.1,0.4,-0.6,0.0,0.5,0.0 \
--trajectory-duration 4.0
```

Listing 3.8: Coordinated motion for selected robots.

Chapter 4

Room 315 Kinematic Shuttle Layer

4.1 Rail network model

The Room 315 rail system is modeled as a directed graph. CSV files contain the geometry of each one-way segment. The topology and routing rules are stored in:

```
mfja_robot_control_config/config/room_315_kinematics/rail_network.yaml
```

The source rail segments are stored in:

```
mfja_robot_control_config/config/room_315_kinematics/raw_segments/
```

Every segment is traversed forward only, from CSV `index=0` to the last row. Routing is not inferred from exact endpoint equality. Routing is decided by explicit tables in `rail_network.yaml`. If the shuttle reaches a segment end and no valid successor exists, it enters `FALLING` mode instead of teleporting or silently correcting the route.

4.2 Current start slots

Only four start slots are currently accepted:

Slot	Gazebo pose	Notes
1	-14.95 -3.86 0.84 0 0 3.14	upper rail
2	-15.43 -3.86 0.84 0 0 3.14	upper rail
3	-15.24 -5.54 0.84 0 0 0	lower rail
4	-14.77 -5.54 0.84 0 0 0	lower rail

Runtime add commands are rejected if the selected start slot is occupied. This is different from collision avoidance: the shuttle is not created at all when the requested start slot is blocked.

4.3 Starting the shuttle node

The same node works in Room 315 only and in the full floor. The key parameter is `gazebo_world_name`.

```

cd /home/tiago/ALI_ros2_ws
source /opt/ros/jazzy/setup.bash
source install/setup.bash

ros2 run mfja_robot_control_config room_315_kinematic_shuttle_node.py --ros-args \
  -p gazebo_world_name:=room_315_only \
  -p shuttle_count:=4 \
  -p start_slots:=1,2,3,4 \
  -p enable_gazebo_set_pose:=true \
  -p enable_gazebo_spawn:=true \
  -p speed:=0.2 \
  -p gazebo_set_pose_rate_hz:=10.0

```

Listing 4.1: Starting four shuttles in Room 315 only.

```

ros2 run mfja_robot_control_config room_315_kinematic_shuttle_node.py --ros-args \
  -p gazebo_world_name:=mfja_3rd_floor \
  -p shuttle_count:=4 \
  -p start_slots:=1,2,3,4 \
  -p enable_gazebo_set_pose:=true \
  -p enable_gazebo_spawn:=true \
  -p speed:=0.2 \
  -p gazebo_set_pose_rate_hz:=10.0

```

Listing 4.2: Starting the same shuttle node in the full floor.

4.4 Switch primitive

Switch routing is controlled by:

`/room_315/switch_states`

The switch primitive has two logical states:

- G: big-loop branch;
- S: small-loop branch.

Aliases such as **BIG**, **LARGE**, and **SMALL** are also accepted by the command layer.

```

ros2 topic pub --once /room_315/switch_states std_msgs/msg/String "{data: 'ALL=G'}"
ros2 topic pub --once /room_315/switch_states std_msgs/msg/String "{data: 'ALL=S'}"
ros2 topic pub --once /room_315/switch_states std_msgs/msg/String "{data: 'A1=G A2=S A3=G A4=S'}"

```

Listing 4.3: Switch commands.

4.5 Stopper primitive

Stopper logic is independent from switch logic. A stopper is a binary primitive:

- 0, OPEN, RELEASE, OFF: the stopper is open and shuttles may pass;
- 1, STOP, CLOSED, ON: the stopper stops a shuttle before the switch.

The stopper command topic is:

`/room_315/stopper_states`

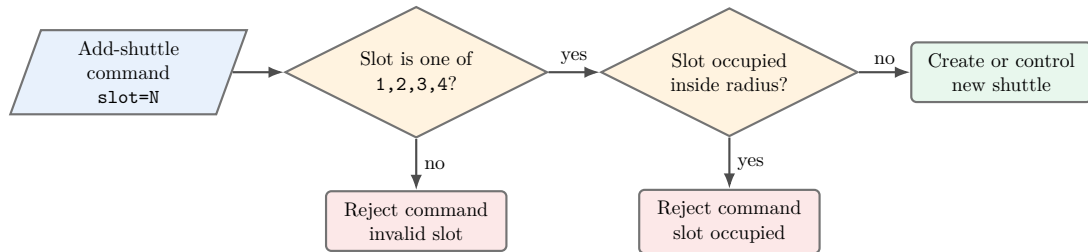


Figure 4.1: Start-slot acceptance logic. A shuttle is added only when the requested slot exists and the start position is clear.

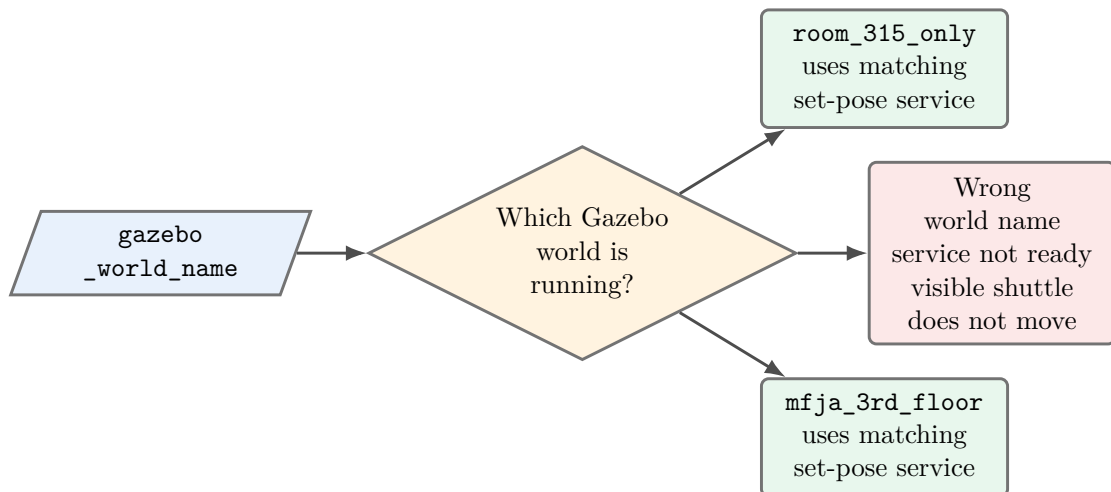


Figure 4.2: Gazebo world-name selection. The shuttle node can publish ROS topics without Gazebo moving anything if the set-pose service name does not match the running world.

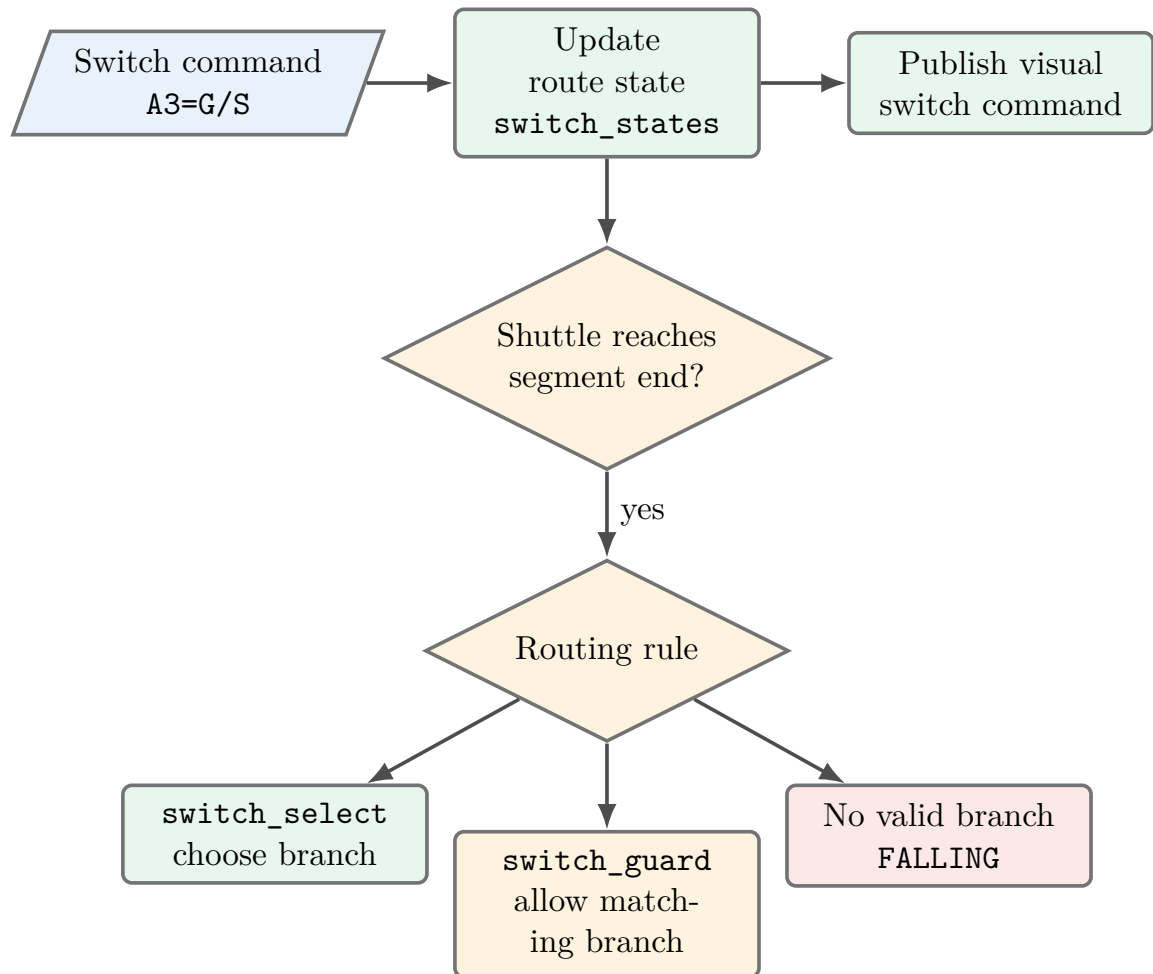


Figure 4.3: Switch command flow. The same command updates route logic and the visual switch. The selected route is used only when the shuttle reaches a segment boundary.

The current logical stoppers are:

Stopper	Before switch	Stop segments
A1	A1	A14
A2	A2	A12E, A12I
A3	A3	A23
A4	A4	A34E, A34I

```
ros2 topic pub --once /room_315/stopper_states std_msgs/msg/String "{data: 'A1=1'}"
ros2 topic pub --once /room_315/stopper_states std_msgs/msg/String "{data: 'A1=0'}"
ros2 topic pub --once /room_315/stopper_states std_msgs/msg/String "{data: 'ALL=1'}"
ros2 topic pub --once /room_315/stopper_states std_msgs/msg/String "{data: 'ALL=0'}"
```

Listing 4.4: Stopper commands.

4.6 Sensor workflow

The educational workflow required by the project is:

sensor -> stop shuttle -> move switch -> unstop shuttle

The sensor-style approach events are published on:

/room_315/sensors/switch_approach

```
ros2 topic echo /room_315/sensors/switch_approach
```

Listing 4.5: Watching approach sensor events.

Each message is JSON text. The field **sensors** is a list because several shuttles can trigger approach sensors at the same time. The field **entity_name** identifies which shuttle the event belongs to.

```

{
  "sensors": [
    {
      "before_switch": "A3",
      "distance_m": 0.247,
      "entity_name": "room315_shuttle_4",
      "segment": "A23",
      "sensor": "A3_APPROACH",
      "stopper": "A3",
      "stopper_state": "0"
    }
  ]
}

```

Listing 4.6: Example approach sensor event.

This means that `room315_shuttle_4` is on segment `A23`, approaching switch `A3`, and is about `0.247` m before the `A3` stop point. If the distance decreases over time, the shuttle is moving toward that stopper.

4.7 Manual stop-switch-release workflow

```

ros2 topic echo /room_315/sensors/switch_approach
ros2 topic pub --once /room_315/stopper_states std_msgs/msg/String "{data: 'A3=1'}"
ros2 topic pub --once /room_315/switch_states std_msgs/msg/String "{data: 'A3=S'}"
ros2 topic pub --once /room_315/stopper_states std_msgs/msg/String "{data: 'A3=0'}"

```

Listing 4.7: Manual stopper and switch workflow.

The stopper stops the shuttle before the switch. The switch can then be moved while the shuttle is waiting. Opening the stopper releases the shuttle so it continues according to the new switch state.

4.8 Per-shuttle ON/OFF control

Each shuttle can be independently enabled or disabled through:

`/room_315/shuttle/control_cmd`

Disabling a shuttle keeps the model in place and stops its kinematic motion. Enabling it again lets it continue from the current rail segment and arc-length position.

```

ros2 topic pub --once /room_315/shuttle/control_cmd std_msgs/msg/String "{data: 'room315_shuttle_2=OFF'}"
ros2 topic pub --once /room_315/shuttle/control_cmd std_msgs/msg/String "{data: 'room315_shuttle_2=ON'}"
ros2 topic pub --once /room_315/shuttle/control_cmd std_msgs/msg/String "{data: 'ALL=OFF'}"
ros2 topic pub --once /room_315/shuttle/control_cmd std_msgs/msg/String "{data: 'ALL=ON'}"

```

Listing 4.8: Per-shuttle enable and disable commands.

4.9 Runtime add behavior

New shuttles can be requested through:

`/room_315/shuttle/add_cmd`

```
ros2 topic pub --once /room_315/shuttle/add_cmd std_msgs/msg/String "{data: 'slot=3'}"  
ros2 topic pub --once /room_315/shuttle/add_cmd std_msgs/msg/String "{data: 'entity=room315_shuttle_5  
slot=3 speed=0.2'}"
```

Listing 4.9: Runtime shuttle add commands.

If the requested start slot is occupied, the node rejects the command and does not create a new shuttle. This behavior was chosen because the expected operational rule is explicit refusal, not silent insertion followed by collision avoidance.

4.10 Collision avoidance

Collision avoidance remains separate from start-slot rejection and from stoppers. It is enabled by default:

- `enable_collision_avoidance=true`
- `shuttle_collision_distance_m=0.33`

If a moving shuttle gets too close to another active shuttle, it enters `WAITING` at the last safe pose instead of merging through the other shuttle.

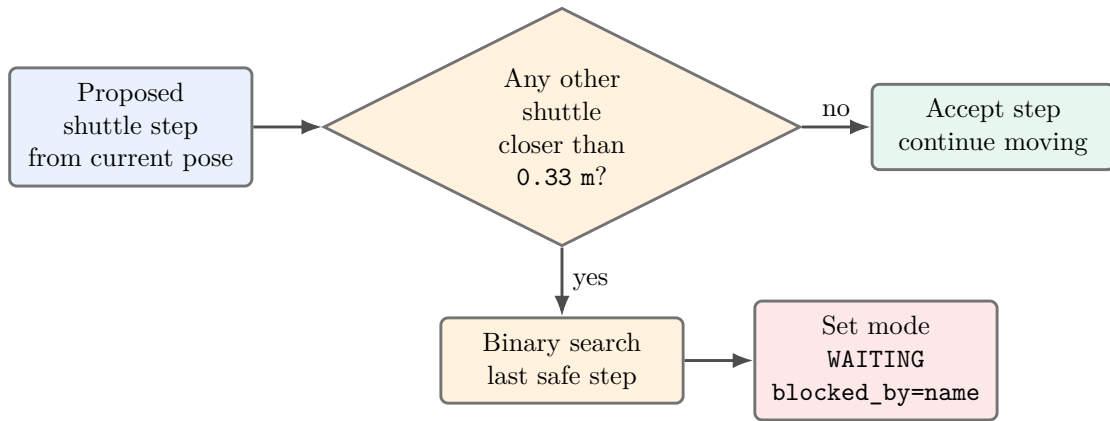


Figure 4.4: Collision avoidance logic. The node does not use contact dynamics in this phase; it prevents overlap by stopping at the last safe kinematic pose.

Chapter 5

Validation Procedures

5.1 CSV and graph validation

```
cd /home/tiago/ALI_ros2_ws
source /opt/ros/jazzy/setup.bash
source install/setup.bash

ros2 run mfja_robot_control_config room_315_csv_preprocessor.py
ros2 run mfja_robot_control_config room_315_network_validator.py
ros2 run mfja_robot_control_config room_315_continuous_path_validator.py
ros2 run mfja_robot_control_config room_315_segment_plot.py
```

Listing 5.1: Room 315 preprocessing and validation commands.

The expected validation result is:

```
Status: PASS (0 warnings)
```

5.2 Runtime topics to inspect

```
ros2 topic echo /room_315/shuttle/state --once
ros2 topic echo /room_315/sensors/switch_approach --once
ros2 topic echo /room_315/shuttle/pose_cmd --once
ros2 topic echo /room_315/shuttles/room315_shuttle_3/pose_cmd --once
ros2 topic echo /mfja/conveyor/switch_states --once
ros2 topic list | grep room_315
```

Listing 5.2: Runtime state and debug topics.

In the shuttle state output, the most important fields are:

- `entity_name`: which shuttle the state describes;
- `mode`: MOVING, WAITING, or FALLING;
- `enabled`: whether the shuttle is allowed to move;
- `stopped_by`: stopper name or DISABLED;
- `blocked_by`: another shuttle that caused collision avoidance;
- `switch_states`: current route switch states;
- `stopper_states`: current stopper states.

5.3 Expected negative tests

```
# This should be rejected if slot 1 is still occupied.  
ros2 topic pub --once /room_315/shuttle/add_cmd std_msgs/msg/String "{data: 'slot=1'}"  
  
# This should always be rejected because only slots 1, 2, 3, and 4 are valid.  
ros2 topic pub --once /room_315/shuttle/add_cmd std_msgs/msg/String "{data: 'slot=5'}"
```

Listing 5.3: Negative tests for start-slot rules.

The first command should produce an occupied-slot error when a shuttle is still inside the start-slot occupancy radius. The second command should report that the valid slots are 1, 2, 3, and 4.

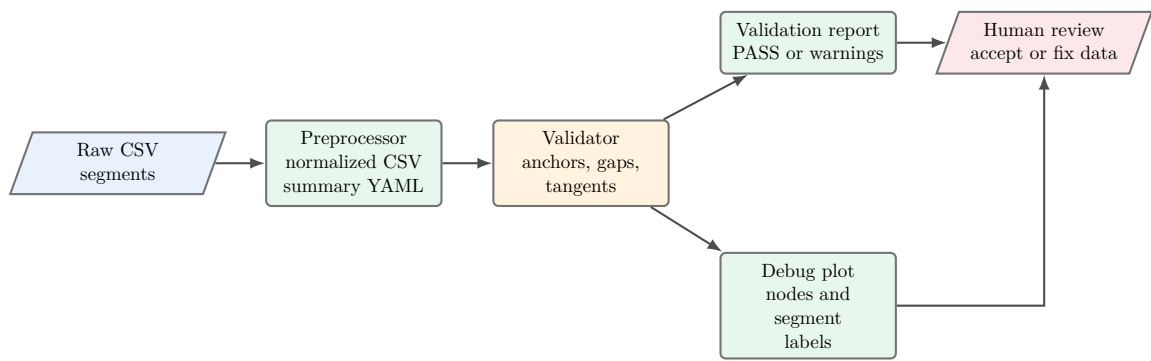


Figure 5.1: Validation workflow. The plots are not decorative; they are the visual check that the CSV geometry, anchors, and directed graph agree with the physical rail layout.

Chapter 6

Current Operational Notes

- Use `gazebo_world_name:=room_315_only` for the isolated Room 315 world.
- Use `gazebo_world_name:=mfja_3rd_floor` for the full-floor world.
- If Gazebo `set_pose` service is not ready yet appears, check the world name and `ros2` service list.
- If a shuttle stops with `stopped_by` set to a stopper name, open that stopper through `/room_315/stopper_states`.
- If a shuttle stops with `stopped_by=DISABLED`, enable it again through `/room_315/shuttle/control_cmd`.
- If a shuttle enters `FALLING`, the graph has no valid successor for the current switch configuration.
- If a switch moves visually but routing does not change, send commands to `/room_315/switch_states`, not directly to the visual switch topic.

Chapter 7

Detailed Implementation Walkthrough

This chapter explains how the current implementation is organized internally. It is written as a code-oriented engineering walkthrough, not only as an operator guide. The goal is to make the current system understandable enough that another developer can maintain it, extend it, or debug it during a live demonstration.

7.1 Design rule: topology is explicit, geometry is only geometry

The most important design decision in Room 315 is that CSV geometry is not used as the routing authority. CSV files describe points along each rail segment. They are used to compute the shuttle pose and yaw at an arc-length position. They are not used to guess which segment comes next by comparing endpoints.

The authoritative route logic lives in `mfja_robot_control_config/config/room_315_kinematics/rail_network.yaml`. That file declares the nodes, segment start and end nodes, switch rules, fixed transitions, stopper locations, and start slots. This prevents fragile behavior caused by tiny endpoint mismatches in the CSV files.

7.2 How the rail paths were built from CSV files

The Room 315 shuttle path was not drawn as one large loop in code. It was built as a measured, directed rail network from CSV segment files. The CSV files are the geometric input. The YAML network is the topological input. The Python core combines both at runtime.

The complete path-building workflow is:

```
measured rail points
-> CSV files with columns index,x,y,z
-> one file per directed rail segment
-> duplicate-point cleanup and normalization
-> endpoint snapping to named anchors
-> rail_network.yaml topology and routing
-> validation report and debug plots
-> arc-length geometry in the Python core
-> ROS 2 node publishes Gazebo poses
```

Listing 7.1: CSV-to-runtime workflow used for Room 315.

Step 1: split the physical rail into directed segments

The physical rail was split at the switch breakpoints. Each piece between two important connection points became one segment. The current accepted segment set is:

```
A1G A1S A2G A2S A3G A3S A4G A4S A12E A12I A14 A23 A34E A34I
```

Every segment has its own CSV file. Each CSV uses this schema:

```
index,x,y,z
0,-12.411...,...
1,-12.412...,...
...
```

The `index` column preserves the intended travel order. The shuttle travels only from `index=0` toward the last row of that file. That rule is important because the same physical rail shape could otherwise be interpreted in two directions. The current network does not allow reverse traversal.

Step 2: keep CSV geometry separate from routing

The CSV rows are used to compute where the shuttle is on a segment. They do not decide which segment comes next. That decision comes only from `rail_network.yaml`. This separation prevents a common bug: two segment endpoints can be physically connected but not numerically identical. If routing used exact endpoint equality, tiny measurement or export errors could break the graph.

The CSV geometry answers this question:

```
At distance s along segment A23, where is the shuttle pose?
```

The topology answers this question:

```
When A23 ends, which segment is valid under the current A3 switch state?
```

Step 3: preprocess and normalize the CSV files

The preprocessing step reads the raw segment CSV files, removes duplicate consecutive points, preserves the directed point order, and writes normalized CSV files. Duplicate points are removed because a zero-length edge has no useful direction and can create unstable yaw values.

The preprocessing command is:

```
ros2 run mfja_robot_control_config room_315_csv_preprocessor.py
```

The relevant inputs and outputs are:

- raw CSV input: `mfja_robot_control_config/config/room_315_kinematics/raw_segments`;
- normalized CSV output: `mfja_robot_control_config/config/room_315_kinematics/normalized_segments`;
- segment summary: `mfja_robot_control_config/config/room_315_kinematics/segment_summary.yaml`.

Step 4: create explicit node anchors

After the geometry exists, endpoints are matched to named anchors. The current network has twelve anchors:

```
A1_C A1_G A1_S
A2_C A2_G A2_S
A3_C A3_G A3_S
A4_C A4_G A4_S
```

The suffix meaning is:

- C: switch center or common connection point;
- G: endpoint on the G branch;
- S: endpoint on the S branch.

The validator snaps segment endpoints to these anchors using a tolerance of 0.05 m. Snapping here means validation association, not moving the runtime shuttle by magic. The association says: this segment endpoint belongs to this named node if it is close enough.

Step 5: define the graph and switch behavior

After endpoints are attached to anchors, the YAML file defines how the directed graph behaves. Fixed connections are simple. Switches are explicit. Converging switches use guard logic so the shuttle fails if it approaches a branch that is not selected.

The three rule types are:

- **fixed**: always move to the same next segment;
- **switch_select**: choose the outgoing branch from the switch state;
- **switch_guard**: allow entry only if the switch state matches the incoming branch.

This is why the model is more precise than a simple "big loop/small loop" toggle. The current state of each switch affects only the future route when the shuttle reaches that switch. It does not teleport the shuttle or rewrite the segment it is already on.

Step 6: validate visually and numerically

The network validator checks the geometry and the topology together:

- segment lengths;
- removed duplicate counts;
- endpoint-to-anchor distances;
- gaps above 0.05 m;
- tangent mismatches above 25 deg;
- whole-network debug plots with segment and node labels.

The validation command is:

```
ros2 run mfja_robot_control_config room_315_network_validator.py
```

The expected result for the current data is:

```
Status: PASS (0 warnings)
```

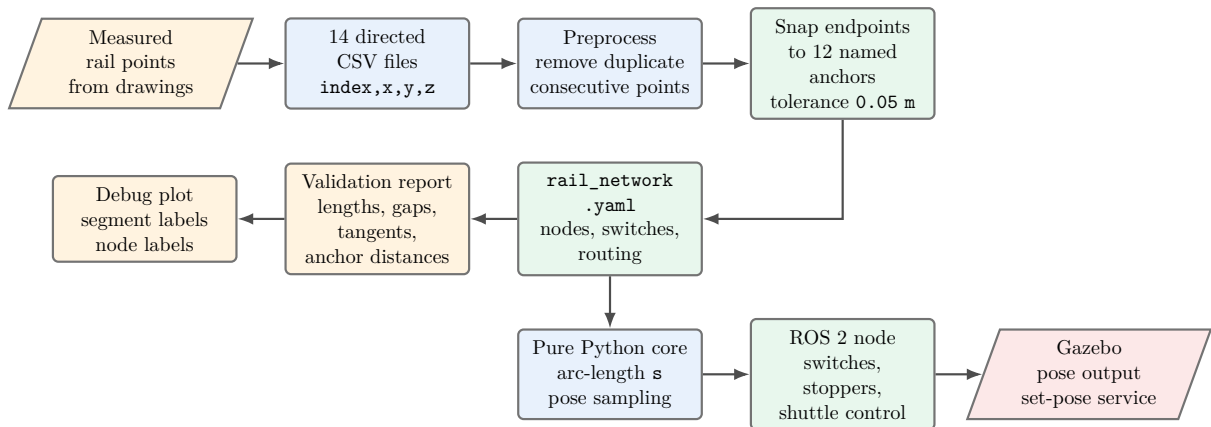


Figure 7.1: End-to-end flow chart for converting measured Room 315 rail data into a running kinematic shuttle path. CSV files provide geometry. The YAML file provides topology and routing. The ROS node combines both and publishes Gazebo poses.

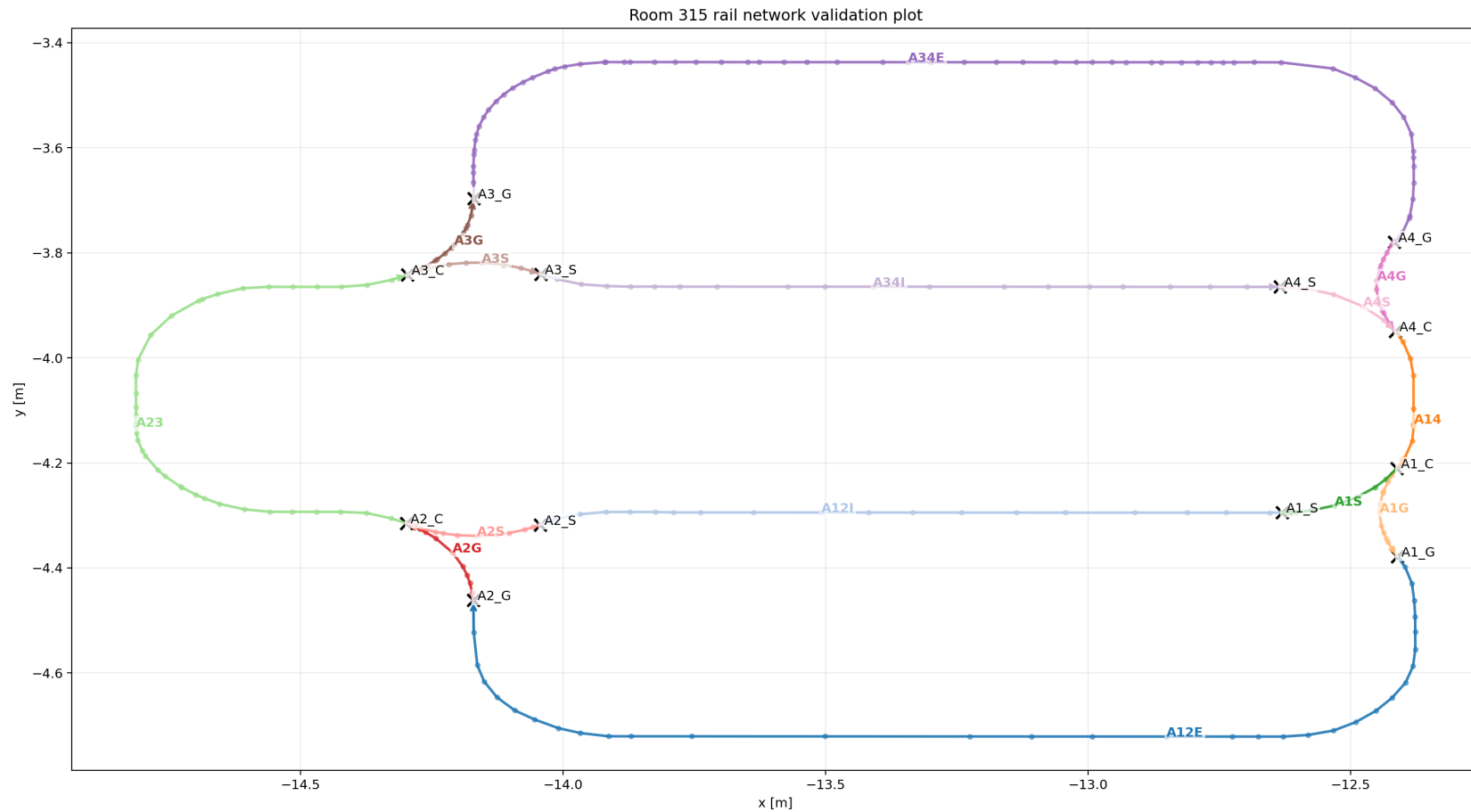


Figure 7.2: Room 315 directed rail network validation plot. Segment labels show the directed CSV segments. Node labels such as A1_C, A1_G, and A1_S show the explicit anchors used by the topology layer.

Step 7: use arc length during simulation

At runtime, the Python core converts each CSV polyline into cumulative arc-length values. If the shuttle state says `current_segment=A23` and `s=0.40`, the core samples the point located 0.40 m along A23. Yaw is computed from the local CSV edge direction.

This gives the shuttle a continuous motion variable s , even though the path is stored as discrete CSV rows. The shuttle never jumps from CSV point to CSV point as a state machine. It interpolates along the polyline.

Step 8: apply the route only at segment boundaries

Switch states are checked when the shuttle reaches a segment end. Changing a switch while the shuttle is in the middle of a segment changes only the future successor. It does not move the shuttle to another branch immediately. This is the behavior needed for a realistic kinematic-first model.

For example:

```
Shuttle is on A23.  
A3 changes from G to S.  
Shuttle stays on A23.  
When A23 ends, the routing table selects A3S instead of A3G.
```

Step 9: bake accepted calibration into the CSV data

During calibration, runtime scale and offset parameters were used to align the shuttle path with the visible rail in Gazebo. Once the correct values were found, the CSV coordinates were updated so normal operation no longer requires manual scale and offset parameters. This makes startup simpler and less error prone.

The normal node command should therefore use default pose scale and offset. Runtime calibration parameters remain available only for future measurement tests.

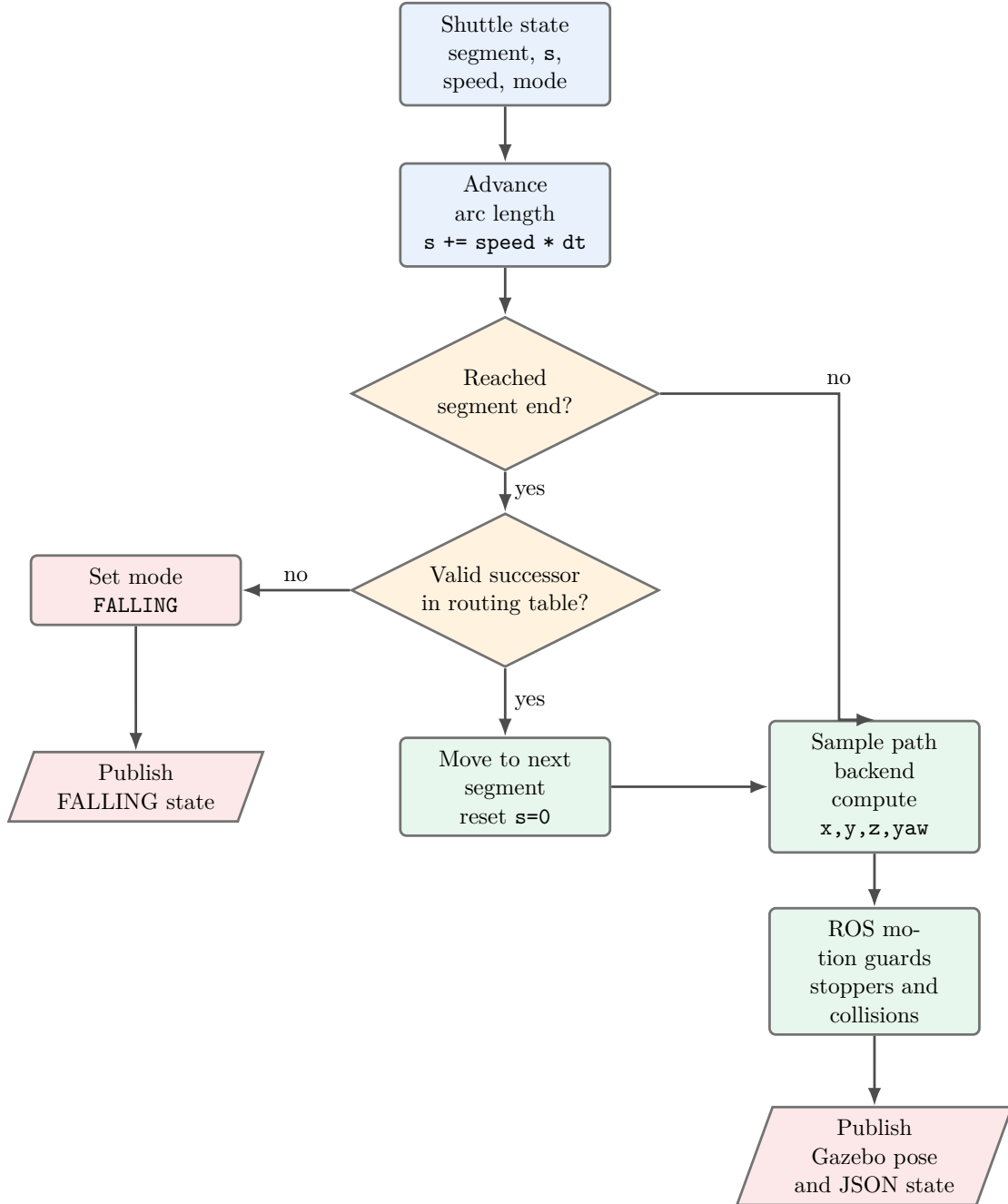


Figure 7.3: Runtime pose computation flow. The selected path backend is sampled by arc length for geometry, while the routing table is consulted only when a segment ends.

```

1  schema_version: 2
2  network_id: room_315_directed_one_way
3  description: 'Explicit directed rail topology for the room 315 shuttle network. Every
4    segment is traversed forward-only, from CSV index 0 to the last row.
5
6    '
7  source_csv_dir: mfja_robot_control_config/config/room_315_kinematics/raw_segments
8  normalized_csv_dir: mfja_robot_control_config/config/room_315_kinematics/normalized_segments
9  snap_tolerance_m: 0.05
10 motion_policy:
11   segment_direction: forward_only
12   traversal_rule: from_csv_index_0_to_last
13   reverse_traversal_allowed: false
14   on_missing_successor: FALLING
15   topology_source_of_truth: explicit_nodes_and_routing_tables
16 switch_state_space:
17   values:
18   - G
19   - S
20   note: 'Switch states match the segment suffixes. G selects the G branch, and S selects
21     the S branch. A later command layer may expose LEFT/RIGHT aliases if desired.
22
23   '
24 stopper_state_space:
25   values:
26   - '0'
27   - '1'
28   note: 'Stopper states are independent from switch states. 0 means open/released,
29     and 1 means stop/closed.
30
31   '

```

Listing 7.2: Top-level Room 315 network policy and state spaces.

The relevant behavior encoded here is:

- all rail segments are directed and forward-only;
- a segment is traversed from CSV index 0 to the last CSV row;
- reverse traversal is explicitly disabled;
- if no valid successor exists, the shuttle enters **FALLING**;
- switch states are independent from stopper states;
- topology and routing tables are the source of truth.

This is why the system does not silently auto-correct a bad path. If a switch state makes the current rail branch invalid, the shuttle core reports a failure state. That failure is useful during validation because it exposes topology mistakes immediately instead of hiding them behind automatic rerouting.

7.3 Start slots and physical entry poses

The current system intentionally exposes only four valid start slots. These are the four entry locations that were accepted for the current Room 315 phase. The same values appear in both the YAML configuration and the Python fallback constants. The YAML version is the preferred configuration source. The Python constants keep the node usable if an older network file does not yet contain `start_slots`.

```

80 @dataclass(frozen=True)
81 class AllowedStartPose:
82     x: float
83     y: float
84     z: float
85     roll: float
86     pitch: float
87     yaw: float
88
89
90 ALLOWED_START_POSES = {
91     '1': AllowedStartPose(-14.95, -3.86, 0.84, 0.0, 0.0, 3.14),
92     '2': AllowedStartPose(-15.43, -3.86, 0.84, 0.0, 0.0, 3.14),
93     '3': AllowedStartPose(-15.24, -5.54, 0.84, 0.0, 0.0, 0.0),
94     '4': AllowedStartPose(-14.77, -5.54, 0.84, 0.0, 0.0, 0.0),
95 }
96
97
98 @dataclass(frozen=True)
99 class StopPoint:
100     segment: str
101     stop_s: float
102     sensor_distance_m: float
103
104
105 @dataclass(frozen=True)
106 class StopperConfig:
107     name: str
108     before_switch: str
109     default_state: str
110     stop_points: tuple[StopPoint, ...]
111
112
113 @dataclass
114 class ManagedShuttle:
115     entity_name: str
116     start_slot: str
117     start_pose: AllowedStartPose
118     start_snap_distance_m: float
119     core: KinematicShuttleCore
120     pose_publisher: object
121     pending_set_pose: object | None = None
122     pending_spawn: object | None = None
123     last_gazebo_set_pose_time: object | None = None
124     gazebo_spawned: bool = False
125     blocked_by: str | None = None
126     collision_distance_m: float | None = None
127     enabled: bool = True
128     stopped_by: str | None = None
129     stopper_distance_m: float | None = None
130     set_pose_warning_logged: bool = False

```

Listing 7.3: Start-slot fallback constants and per-shuttle runtime state.

Each start slot contains a Gazebo pose in the order:

```
x y z roll pitch yaw
```

The current four slots are:

Slot	x	y	z	roll	pitch	yaw
1	-14.95	-3.86	0.84	0.0	0.0	3.14
2	-15.43	-3.86	0.84	0.0	0.0	3.14
3	-15.24	-5.54	0.84	0.0	0.0	0.0
4	-14.77	-5.54	0.84	0.0	0.0	0.0

At startup, the node does not blindly put a shuttle on the pose and call it valid. It

snaps the requested start pose to the nearest point on the rail network. The log message includes the selected segment, arc-length value, and snap distance. A good startup line looks like this:

```
start_slot=2, start_segment=A34E, start_s=1.455, start_snap_distance=0.005 m
```

The snap distance is important. It tells the operator how far the configured Gazebo start pose is from the rail geometry. A small value means the slot is aligned with the rail. A large value means the configured slot should be reviewed.

7.4 ROS node parameter model

The main runtime node is `mfja_robot_control_config/scripts/room_315_kinematic_shuttle_node.py`. It is intentionally parameter-driven so the same executable can be used in the Room-only world and the full-floor world.

```

137
138 self.declare_parameter('network_yaml', str(default_network_path()))
139 self.declare_parameter('path_backend', CUBIC_HERMITE_PATH_BACKEND)
140 self.declare_parameter('arc_length_samples_per_edge', 16)
141 self.declare_parameter('shuttle_count', 1)
142 self.declare_parameter('start_slot', 2)
143 self.declare_parameter('start_slots', '')
144 self.declare_parameter('start_snap_tolerance_m', 0.25)
145 self.declare_parameter('initial_segment', 'A14')
146 self.declare_parameter('initial_s', 0.0)
147 self.declare_parameter('speed', 0.25)
148 self.declare_parameter('update_rate_hz', 30.0)
149 self.declare_parameter('enable_collision_avoidance', True)
150 self.declare_parameter('shuttle_collision_distance_m', 0.33)
151 self.declare_parameter('collision_search_iterations', 12)
152 self.declare_parameter('pose_topic', '/room_315/shuttle/pose_cmd')
153 self.declare_parameter('pose_topic_prefix', '/room_315/shuttles')
154 self.declare_parameter('state_topic', '/room_315/shuttle/state')
155 self.declare_parameter('add_shuttle_command_topic', '/room_315/shuttle/add_cmd')
156 self.declare_parameter('shuttle_control_command_topic', '/room_315/shuttle/control_cmd')
157 self.declare_parameter('switch_command_topic', '/room_315/switch_states')
158 self.declare_parameter('stopper_command_topic', '/room_315/stopper_states')
159 self.declare_parameter('sensor_state_topic', '/room_315/sensors/switch_approach')
160 self.declare_parameter('pose_offset_command_topic', '/room_315/shuttle/pose_offset_cmd')
161 self.declare_parameter('visual_switch_command_topic', '/mfja/conveyor/switch_cmd')
162 self.declare_parameter('visual_switch_state_topic', '/mfja/conveyor/switch_states')
163 self.declare_parameter('sync_from_visual_switch_states', True)
164 self.declare_parameter('frame_id', 'world')
165 self.declare_parameter('enable_gazebo_set_pose', False)
166 self.declare_parameter('gazebo_world_name', 'room_315_only')
167 self.declare_parameter('gazebo_set_pose_service', '')
168 self.declare_parameter('enable_gazebo_spawn', True)
169 self.declare_parameter('gazebo_spawn_service', '')
170 self.declare_parameter('shuttle_model_sdf', str(default_shuttle_model_sdf_path()))
171 self.declare_parameter('preloaded_shuttle_count', 4)
172 self.declare_parameter('reject_occupied_start_slots', True)
173 self.declare_parameter('start_slot_occupancy_radius_m', 0.33)
174 self.declare_parameter('gazebo_entity_name', 'room315_shuttle_1')
175 self.declare_parameter('gazebo_entity_names', '')
176 self.declare_parameter('gazebo_set_pose_rate_hz', 10.0)
177 self.declare_parameter('publish_visual_switch_commands', True)
178 self.declare_parameter('enable_gazebo_pose_transform', True)
179 self.declare_parameter('pose_transform_a', -0.893249246800)
180 self.declare_parameter('pose_transform_b', 0.005839516878)
181 self.declare_parameter('pose_transform_tx', -26.921427375871)
182 self.declare_parameter('pose_transform_c', 0.001889497475)
183 self.declare_parameter('pose_transform_d', 1.308619216904)
184 self.declare_parameter('pose_transform_ty', 0.666926143808)
185 self.declare_parameter('pose_transform_z_offset', 0.0)
186 self.declare_parameter('pose_transform_yaw_offset', 0.0)
187 self.declare_parameter('pose_scale_x', 1.0)
188 self.declare_parameter('pose_scale_y', 1.0)
189 self.declare_parameter('pose_scale_origin_x', -15.855195431322)
190 self.declare_parameter('pose_scale_origin_y', -4.525523413467)

```

Listing 7.4: Main ROS node parameters for the kinematic shuttle system.

The key parameter groups are:

- network and geometry parameters: `network_yaml`, `path_backend`, `arc_length_samples_per_edge`, and `start_snap_tolerance_m`;
- shuttle initialization: `shuttle_count`, `start_slot`, `start_slots`, `gazebo_entity_name`, `gazebo_entity_names`;
- motion timing: `speed`, `update_rate_hz`, `gazebo_set_pose_rate_hz`;
- safety behavior: `enable_collision_avoidance`, `shuttle_collision_distance_m`, `reject_occupied_start_slots`;

- command topics: switch, stopper, shuttle add, shuttle ON/OFF, pose offset;
- visual integration topics for the physical switch meshes;
- Gazebo integration: world name, set-pose service, spawn service, and SDF path;
- pose transform and calibration parameters.

The default collision distance is 0.33 m. It is enabled by default and does not need to be written in normal terminal commands. This value was chosen as the current practical stopping distance for the shuttle model in the kinematic simulation.

7.5 Loading start slots from the network file

The node loads allowed start slots from `rail_network.yaml`. If the YAML file has no `start_slots` section, the code falls back to the Python constants shown earlier.

```

421     def _resolve_world_service(self, raw_service: str, suffix: str) -> str:
422         service = raw_service.strip()
423         if service:
424             return service
425
426         world_name = self.gazebo_world_name.strip().strip('/')
427         if not world_name:
428             raise ValueError(
429                 'gazebo_world_name cannot be empty when Gazebo service names are auto-derived.'
430             )
431         return f'/world/{world_name}/{suffix}'
432
433     def _load_allowed_start_poses(self) -> Dict[str, AllowedStartPose]:
434         raw_slots = self.network.config.get('start_slots') or {}
435         if not raw_slots:
436             return dict(ALLOWED_START_POSES)
437
438         allowed: Dict[str, AllowedStartPose] = {}

```

Listing 7.5: Allowed start-slot loading from `rail_network.yaml`.

This keeps configuration ownership clear: the network file owns the operational start slots, and the node only provides defaults for compatibility.

7.6 Stopper configuration and stop-point generation

The stoppers are independent from switches. Each stopper is attached to the approach side of a switch. A stopper does not decide the route. It only decides whether an approaching shuttle must stop before the switch.

```

24 stopper_state_space:
25   values:
26   - '0'
27   - '1'
28   note: 'Stopper states are independent from switch states. 0 means open/released,
29         and 1 means stop/closed.
30
31   '
32 stoppers:
33   A1:
34     before_switch: A1
35     default_state: '0'
36     stop_offset_m: 0.08
37     sensor_distance_m: 0.25
38     segments:
39     - A14
40   A2:
41     before_switch: A2
42     default_state: '0'
43     stop_offset_m: 0.08
44     sensor_distance_m: 0.25
45     segments:
46     - A12E
47     - A12I
48   A3:
49     before_switch: A3
50     default_state: '0'
51     stop_offset_m: 0.08
52     sensor_distance_m: 0.25
53     segments:
54     - A23
55   A4:
56     before_switch: A4
57     default_state: '0'
58     stop_offset_m: 0.08
59     sensor_distance_m: 0.25
60     segments:
61     - A34E
62     - A34I

```

Listing 7.6: Stopper definitions in the Room 315 network file.

The YAML fields mean:

- `before_switch`: the switch protected by this stopper;
- `default_state`: 0 means released, 1 means closed;
- `stop_offset_m`: distance before the end of the approach segment;
- `sensor_distance_m`: distance window where sensor events are published;
- `segments`: approach segments where the stopper can stop a shuttle.

The node converts each configured segment into a concrete stop point. If the YAML does not specify an absolute arc-length `s`, the node computes:

```
stop_s = segment.length - stop_offset_m
```

```

440         slot = str(raw_slot).strip()
441         pose_values = raw_config.get('pose') if isinstance(raw_config, dict) else raw_config
442         if pose_values is None or len(pose_values) != 6:
443             raise ValueError(
444                 f'start_slots.{slot} must define pose: [x, y, z, roll, pitch, yaw].')
445         )
446         allowed[slot] = AllowedStartPose(*[float(value) for value in pose_values])
447
448     if not allowed:
449         raise ValueError('rail_network.yaml start_slots must not be empty.')
450     return allowed
451
452 def _load_stopper_configs(self) -> Dict[str, StopperConfig]:
453     configs: Dict[str, StopperConfig] = {}
454     raw_configs = self.network.config.get('stoppers', {}) or {}
455     for raw_name, raw_config in raw_configs.items():
456         name = str(raw_name).strip().upper()
457         before_switch = str(raw_config.get('before_switch', name)).strip().upper()
458         default_state = self._normalize_stopper_state(
459             str(raw_config.get('default_state', '0')))
460         )
461         default_sensor_distance_m = float(raw_config.get('sensor_distance_m', 0.25))
462         default_stop_offset_m = float(raw_config.get('stop_offset_m', 0.08))
463         stop_points: list[StopPoint] = []
464
465         raw_stop_points = raw_config.get('stop_points')
466         if raw_stop_points is None:
467             raw_stop_points = [
468                 {'segment': segment_name}
469                 for segment_name in raw_config.get('segments', [])
470             ]
471
472         for raw_stop_point in raw_stop_points:
473             segment_name = str(raw_stop_point['segment']).strip()
474             if segment_name not in self.network.segments:
475                 raise ValueError(
476                     f'Stopper {name} references unknown segment {segment_name!r}.')
477             )
478
479             segment = self.network.segments[segment_name]
480             if 's' in raw_stop_point:
481                 stop_s = float(raw_stop_point['s'])
482             else:
483                 stop_offset_m = float(
484                     raw_stop_point.get('stop_offset_m', default_stop_offset_m)
485                 )
486                 stop_s = segment.length - stop_offset_m
487             stop_s = max(0.0, min(stop_s, segment.length))
488             sensor_distance_m = float(
489                 raw_stop_point.get('sensor_distance_m', default_sensor_distance_m)
490             )
491             stop_points.append(
492                 StopPoint(
493                     segment=segment_name,
494                     stop_s=stop_s,
495                     sensor_distance_m=max(0.0, sensor_distance_m),

```

Listing 7.7: Runtime loading of stopper definitions and stop-point conversion.

This keeps the YAML readable while still giving the motion core a precise arc-length stop location.

7.7 Switches, fixed transitions, and routing table

Switches are stored separately from fixed transitions. The switch section is human-readable: it explains which node is controlled and which outgoing branch belongs to each state.

```

296 switches:
297   A1:
298     kind: diverging
299     controlled_node: A1_C
300     states:
301     - G
302     - S
303     incoming_segments:
304     - A14
305     outgoing_by_state:
306       G: A1G
307       S: A1S
308   A2:
309     kind: converging_gate
310     controlled_node: A2_C
311     states:
312     - G
313     - S
314     incoming_path_by_state:
315       G:
316         incoming_segment: A12E
317         connector_segment: A2G
318       S:
319         incoming_segment: A12I
320         connector_segment: A2S
321     outgoing_segments:
322     - A23
323   A3:
324     kind: diverging
325     controlled_node: A3_C
326     states:
327     - G
328     - S
329     incoming_segments:
330     - A23
331     outgoing_by_state:
332       G: A3G
333       S: A3S
334   A4:
335     kind: converging_gate
336     controlled_node: A4_C
337     states:
338     - G
339     - S
340     incoming_path_by_state:
341       G:
342         incoming_segment: A34E
343         connector_segment: A4G
344       S:
345         incoming_segment: A34I
346         connector_segment: A4S
347     outgoing_segments:
348     - A14
349 fixed_transitions:
350   A1G: A12E
351   A1S: A12I
352   A2G: A23
353   A2S: A23
354   A3G: A34E
355   A3S: A34I
356   A4G: A14
357   A4S: A14

```

Listing 7.8: Switch declarations and fixed transitions.

The runtime core does not infer route behavior from names. It reads the `routing_table`. The routing table is the executable graph policy.

```

358 routing_table:
359   A14:
360     type: switch_select
361     switch: A1
362     by_state:
363       G: A1G
364       S: A1S
365     on_unknown_state: FALLING
366   A1G:
367     type: fixed
368     next_segment: A12E
369   A1S:
370     type: fixed
371     next_segment: A12I
372   A12E:
373     type: switch_guard
374     switch: A2
375     by_state:
376       G: A2G
377       S: FALLING
378     on_unknown_state: FALLING
379   A12I:
380     type: switch_guard
381     switch: A2
382     by_state:
383       G: FALLING
384       S: A2S
385     on_unknown_state: FALLING
386   A2G:
387     type: fixed
388     next_segment: A23
389   A2S:
390     type: fixed

```

Listing 7.9: Routing table excerpt showing select, guard, and fixed rules.

There are three important routing patterns:

- **fixed**: the segment always continues to one known successor;
- **switch_select**: the switch chooses the next branch;
- **switch_guard**: the shuttle may continue only if the switch state matches the branch it is physically approaching.

The guard behavior matters at converging switches. For example, if a shuttle is on A12E, it can enter A2G only when switch A2 is in state G. If A2 is in state S, the routing table returns FALLING. This represents an invalid physical configuration, not an automatic detour.

7.8 Pure Python kinematic core

The pure core is in `mfja_robot_control_config/scripts/room_315_kinematic_shuttle.py`. It has no Gazebo dependency and no ROS dependency. This is deliberate: the network and one-shuttle logic can be tested offline before integrating with simulation services.

```

14 MOVING = 'MOVING'
15 WAITING = 'WAITING'
16 FALLING = 'FALLING'
17 POLYLINE_PATH_BACKEND = 'polyline'
18 CUBIC_HERMITE_PATH_BACKEND = 'cubic_hermite'
19 PATH_BACKENDS = {POLYLINE_PATH_BACKEND, CUBIC_HERMITE_PATH_BACKEND}
20
21
22 @dataclass(frozen=True)
23 class Point3D:
24     x: float
25     y: float
26     z: float
27
28     def distance_to(self, other: 'Point3D') -> float:
29         return math.dist((self.x, self.y, self.z), (other.x, other.y, other.z))
30
31
32 @dataclass
33 class ShuttleState:
34     current_segment: str
35     s: float
36     speed: float
37     mode: str = MOVING
38
39
40 @dataclass(frozen=True)
41 class ShuttlePose:
42     x: float
43     y: float
44     z: float
45     yaw: float

```

Listing 7.10: Core shuttle state and pose data structures.

The shuttle state is intentionally small:

- `current_segment`: the active rail segment;
- `s`: progress along that segment in meters;
- `speed`: scalar speed in meters per second;
- `mode`: MOVING, WAITING, or FALLING.

The pose returned by the core includes the interpolated `x`, `y`, `z`, and `yaw`, plus the current segment and mode.

7.9 Arc-length path backends

CSV rows are still the measured input. They are not discarded. The important change is that the runtime geometry layer now has two path backends behind the same `SegmentGeometry.sample(s)` interface:

- `polyline`: direct piecewise-linear interpolation between CSV points. This is the reference and debug backend because it follows the measurement data exactly.
- `cubic_hermite`: continuous cubic Hermite interpolation with arc-length reparameterization. This is the recommended runtime backend because it gives smoother position and yaw along curves.

The runtime state is still parameterized by arc length s . During each update, the core asks the selected backend for the pose at distance s . This keeps the routing code independent from the geometric representation.

```

48     mode: str
49
50
51 class SegmentGeometry:
52     def __init__(
53         self,
54         name: str,
55         points: Sequence[Point3D],
56         path_backend: str = POLYLINE_PATH_BACKEND,
57         arc_length_samples_per_edge: int = 16,
58     ) -> None:
59         if len(points) < 2:
60             raise ValueError(f'{name} must contain at least two points')
61         self.name = name
62         self.points = list(points)
63         self.path_backend = _normalize_path_backend(path_backend)
64         self.arc_length_samples_per_edge = max(4, int(arc_length_samples_per_edge))
65         self.chord_arc_lengths = self._compute_arc_lengths(self.points)
66         self._tangents = self._compute_tangents()
67         self._arc_map: List[tuple[float, int, float]] = []
68         self.arc_lengths = list(self.chord_arc_lengths)
69         self._length = self.chord_arc_lengths[-1]
70         if self.path_backend == CUBIC_HERMITE_PATH_BACKEND:
71             self.arc_lengths, self._arc_map, self._length = self._build_cubic_arc_map()
72         if self.length <= 0.0:
73             raise ValueError(f'{name} has zero total length')
74
75     @staticmethod
76     def _compute_arc_lengths(points: Sequence[Point3D]) -> List[float]:
77         arc_lengths = [0.0]
78         total = 0.0
79         for previous, current in zip(points, points[1:]):
80             total += previous.distance_to(current)
81             arc_lengths.append(total)
82         return arc_lengths
83
84     @property
85     def length(self) -> float:
86         return self._length
87
88     def sample(self, s: float) -> tuple[Point3D, float]:
89         if self.path_backend == CUBIC_HERMITE_PATH_BACKEND:
90             return self._sample_cubic_by_arc_length(s)
91         return self._sample_polyline(s)
92
93     def _sample_polyline(self, s: float) -> tuple[Point3D, float]:
94         clamped_s = max(0.0, min(s, self.length))
95         for index in range(1, len(self.arc_lengths)):
96             previous_s = self.arc_lengths[index - 1]
97             current_s = self.arc_lengths[index]
98             if clamped_s > current_s and index < len(self.arc_lengths) - 1:
99                 continue
100
101             previous = self.points[index - 1]
102             current = self.points[index]
103             edge_length = current_s - previous_s
104             ratio = 0.0 if edge_length == 0.0 else (clamped_s - previous_s) / edge_length
105             x = previous.x + ratio * (current.x - previous.x)
106             y = previous.y + ratio * (current.y - previous.y)
107             z = previous.z + ratio * (current.z - previous.z)
108             yaw = math.atan2(current.y - previous.y, current.x - previous.x)
109             return Point3D(x=x, y=y, z=z), yaw
110
111         previous = self.points[-2]
112         current = self.points[-1]
113         yaw = math.atan2(current.y - previous.y, current.x - previous.x)
114         return current, yaw
115

```

```

116 def _compute_tangents(self) -> List[Point3D]:
117     tangents: List[Point3D] = []
118     for index, point in enumerate(self.points):
119         if index == 0:
120             previous = point
121             current = self.points[index + 1]
122             previous_s = self.chord_arc_lengths[index]
123             current_s = self.chord_arc_lengths[index + 1]
124         elif index == len(self.points) - 1:
125             previous = self.points[index - 1]
126             current = point
127             previous_s = self.chord_arc_lengths[index - 1]
128             current_s = self.chord_arc_lengths[index]
129         else:
130             previous = self.points[index - 1]
131             current = self.points[index + 1]
132             previous_s = self.chord_arc_lengths[index - 1]

```

Listing 7.11: Segment geometry now supports polyline and continuous cubic Hermite backends.

The accepted policy is:

```

Recommended runtime backend: cubic_hermite
Reference/debug backend:    polyline

```

Keeping both backends is intentional. The continuous backend is better for normal operation, but the polyline backend remains the baseline for debugging. If a future path appears misaligned, the operator can immediately compare `cubic_hermite` against `polyline` to determine whether the problem comes from the fitted path layer or from the original CSV geometry.

The practical improvement already made was to use corrected CSV extraction and then bake the calibration scale and offset into the CSV coordinates. That is why normal node startup no longer needs `pose_scale_x`, `pose_scale_y`, `pose_offset_x`, or `pose_offset_y` for the accepted path alignment.

7.10 Continuous path validation

The continuous backend is validated against the CSV polyline before it is used for demos. The validation tool loads the same `rail_network.yaml` twice: once with `polyline` and once with `cubic_hermite`. It then samples the continuous curve and reports:

- polyline length and continuous length;
- length difference in meters and percent;
- maximum and mean deviation from the CSV polyline;
- endpoint deltas;
- maximum yaw step between sampled continuous points;
- warnings if the continuous path deviates too far from the measured CSV.

```

ros2 run mfja_robot_control_config room_315_continuous_path_validator.py

```

The current accepted result is:

```

Validated continuous paths for 14 segments.
Status: PASS (0 warnings)

```

Continuous path validation: CSV measurement vs parametrized path backend

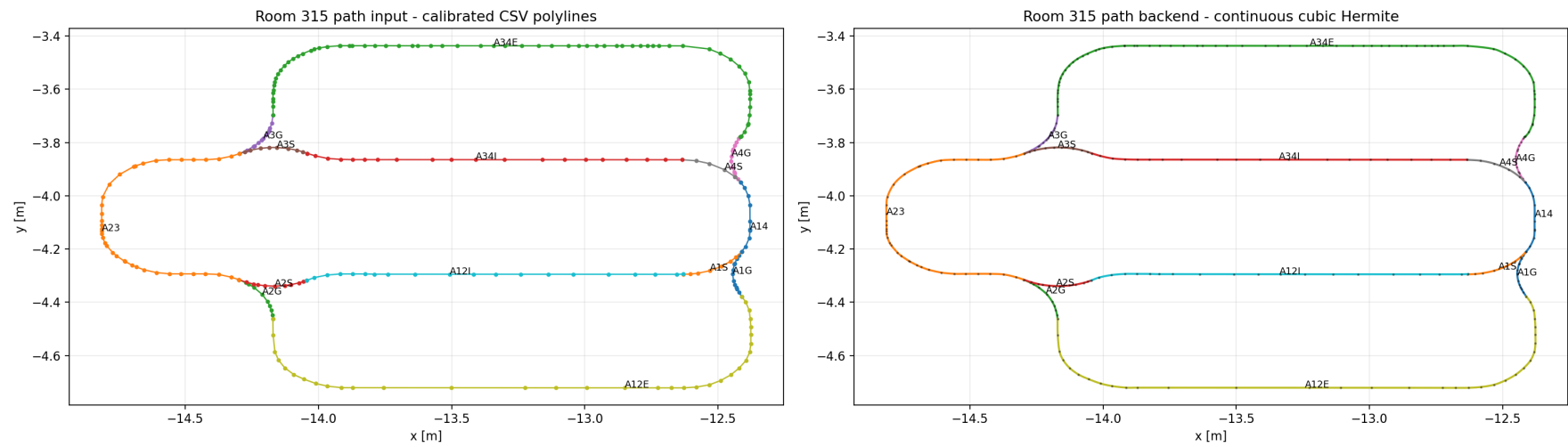


Figure 7.4: Continuous path validation plot. The left side shows the calibrated CSV polylines used as the reference measurement. The right side shows the continuous cubic Hermite backend sampled densely. The black dots on the right are the original CSV points, which makes it easy to detect overshoot.

7.11 Successor resolution and explicit failure

The core asks `RailNetwork.resolve_successor()` what comes after the current segment. If there is no successor, the shuttle enters `FALLING`.

```
95     for index in range(1, len(self.arc_lengths)):
96         previous_s = self.arc_lengths[index - 1]
97         current_s = self.arc_lengths[index]
98         if clamped_s > current_s and index < len(self.arc_lengths) - 1:
99             continue
100
101         previous = self.points[index - 1]
102         current = self.points[index]
103         edge_length = current_s - previous_s
104         ratio = 0.0 if edge_length == 0.0 else (clamped_s - previous_s) / edge_length
105         x = previous.x + ratio * (current.x - previous.x)
106         y = previous.y + ratio * (current.y - previous.y)
107         z = previous.z + ratio * (current.z - previous.z)
108         yaw = math.atan2(current.y - previous.y, current.x - previous.x)
109         return Point3D(x=x, y=y, z=z), yaw
110
111     previous = self.points[-2]
112     current = self.points[-1]
113     yaw = math.atan2(current.y - previous.y, current.x - previous.x)
114     return current, yaw
115
116 def _compute_tangents(self) -> List[Point3D]:
117     tangents: List[Point3D] = []
118     for index, point in enumerate(self.points):
119         if index == 0:
120             previous = point
121             current = self.points[index + 1]
122             previous_s = self.chord_arc_lengths[index]
123             current_s = self.chord_arc_lengths[index + 1]
124         elif index == len(self.points) - 1:
125             previous = self.points[index - 1]
126             current = point
127             previous_s = self.chord_arc_lengths[index - 1]
128             current_s = self.chord_arc_lengths[index]
129         else:
130             previous = self.points[index - 1]
131             current = self.points[index + 1]
132             previous_s = self.chord_arc_lengths[index - 1]
133             current_s = self.chord_arc_lengths[index + 1]
134
135         ds = max(current_s - previous_s, 1e-12)
136         tangents.append(
137             Point3D(
138                 x=(current.x - previous.x) / ds,
139                 y=(current.y - previous.y) / ds,
140                 z=(current.z - previous.z) / ds,
141             )
142         )
143     return tangents
144
145 def _build_cubic_arc_map(self) -> tuple[List[float], List[tuple[float, int, float]], float]:
146     point_arc_lengths = [0.0]
147     arc_map: List[tuple[float, int, float]] = [(0.0, 0, 0.0)]
148     total = 0.0
149
150     for index in range(len(self.points) - 1):
151         previous_point, _ = self._evaluate_cubic_interval(index, 0.0)
152         for sample_index in range(1, self.arc_length_samples_per_edge + 1):
153             u = sample_index / self.arc_length_samples_per_edge
154             current_point, _ = self._evaluate_cubic_interval(index, u)
155             total += previous_point.distance_to(current_point)
156             arc_map.append((total, index, u))
```

Listing 7.12: Rail network loading, switch normalization, and successor lookup.

The function treats `FALLING` as a deliberate result. This is safer than guessing a route

because it tells the operator or developer that the current switch configuration cannot support the current segment.

7.12 Core stepping logic

The step loop advances by `speed * dt`. If the remaining distance reaches the end of the current segment, the core consumes that distance, resolves the successor, and continues on the next segment. One update can cross more than one short segment if the speed and time step allow it.

```

179     dh11 = 3.0 * u2 - 2.0 * u
180
181     point = Point3D(
182         x=h00 * p0.x + h10 * h * m0.x + h01 * p1.x + h11 * h * m1.x,
183         y=h00 * p0.y + h10 * h * m0.y + h01 * p1.y + h11 * h * m1.y,
184         z=h00 * p0.z + h10 * h * m0.z + h01 * p1.z + h11 * h * m1.z,
185     )
186     tangent = Point3D(
187         x=(dh00 * p0.x + dh01 * p1.x) / h + dh10 * m0.x + dh11 * m1.x,
188         y=(dh00 * p0.y + dh01 * p1.y) / h + dh10 * m0.y + dh11 * m1.y,
189         z=(dh00 * p0.z + dh01 * p1.z) / h + dh10 * m0.z + dh11 * m1.z,
190     )
191     return point, tangent
192
193     def _sample_cubic_by_arc_length(self, s: float) -> tuple[Point3D, float]:
194         clamped_s = max(0.0, min(s, self.length))
195         if clamped_s <= 0.0:
196             point, tangent = self._evaluate_cubic_interval(0, 0.0)
197             return point, math.atan2(tangent.y, tangent.x)
198         if clamped_s >= self.length:
199             point, tangent = self._evaluate_cubic_interval(len(self.points) - 2, 1.0)
200             return point, math.atan2(tangent.y, tangent.x)
201
202         low = 0
203         high = len(self._arc_map) - 1
204         while low < high:
205             middle = (low + high) // 2
206             if self._arc_map[middle][0] < clamped_s:
207                 low = middle + 1
208             else:
209                 high = middle
210
211         upper = self._arc_map[low]
212         lower = self._arc_map[max(0, low - 1)]
213         lower_s, lower_interval, lower_u = lower
214         upper_s, upper_interval, upper_u = upper

```

Listing 7.13: Pure kinematic stepping and FALLING behavior.

The core does not know about stoppers, sensors, collision avoidance, Gazebo spawning, or visual switch meshes. Those are ROS-node responsibilities layered around the pure core.

7.13 Runtime shuttle creation

The node supports runtime shuttle addition through `/room_315/shuttle/add_cmd`. The command parser accepts plain slot numbers, assignment strings, and JSON payloads.

```

627     )
628     except (RuntimeError, ValueError, json.JSONDecodeError) as error:
629         self.get_logger().error(f'Failed to add shuttle: {error}')
630         return
631
632     self.shuttles.append(shuttle)
633     self._request_spawn_if_needed(shuttle)
634     self.get_logger().info(
635         f'Added shuttle {shuttle.entity_name} at slot {shuttle.start_slot}; '
636         f'shuttles={self._shuttle_summary()}'
637     )
638
639 def _parse_add_shuttle_command(self, raw_command: str) -> tuple[str, str, float]:
640     command = raw_command.strip()
641     if not command:
642         raise ValueError('Empty add shuttle command')
643
644     if command.startswith('{'):
645         payload = json.loads(command)
646         assignments = [(str(key), str(value)) for key, value in payload.items()]
647     elif '=' not in command and ':' not in command:
648         assignments = [('slot', command)]
649     else:
650         assignments = []
651         for token in re.split(r'[\s,;]+', command.replace(':', '=')):
652             if not token:
653                 continue
654             if '=' not in token:
655                 raise ValueError(
656                     f'Add shuttle command must look like slot=3, got {token!r}'
657                 )
658             key, raw_value = token.split('=', 1)
659             assignments.append((key, raw_value))
660
661     slot = ''
662     entity_name = ''
663     speed = self.default_shuttle_speed
664     for raw_key, raw_value in assignments:
665         key = raw_key.strip().lower()
666         value = raw_value.strip()
667         if key in {'slot', 'start_slot', 'start'}:
668             slot = self._normalize_start_slot(value)
669         elif key in {'entity', 'entity_name', 'gazebo_entity_name', 'name'}:
670             entity_name = value
671         elif key == 'speed':
672             speed = float(value)
673         else:
674             raise ValueError(
675                 f'Unknown add shuttle key {raw_key!r}; use slot, entity, or speed.'
676             )
677
678     if not slot:
679         slot = self._next_unused_start_slot()
680     if self.reject_occupied_start_slots:
681         occupied_by = self._start_slot_occupancy_blocker(slot)
682         if occupied_by is not None:
683             entity_name_at_slot, distance_m = occupied_by
684             raise ValueError(
685                 f'start slot {slot} is occupied by {entity_name_at_slot} '
686                 f'at distance {distance_m:.3f} m; add command rejected.'
687             )
688
689     if not entity_name:
690         entity_name = self._next_unused_entity_name()
691     if any(shuttle.entity_name == entity_name for shuttle in self.shuttles):
692         raise ValueError(
693             f'Gazebo entity {entity_name!r} is already controlled by this node.'
694         )
695
696     return entity_name, slot, speed
697
698 def _next_unused_start_slot(self) -> str:

```

```

699     slots = sorted(self.allowed_start_poses)
700     if not self.reject_occupied_start_slots:
701         return slots[len(self.shuttles) % len(slots)]
702
703     for slot in slots:
704         if self._start_slot_occupancy_blocker(slot) is None:
705             return slot
706     raise ValueError('All allowed start slots are currently occupied.')

```

Listing 7.14: Runtime add-shuttle parser and occupied-slot rejection.

The current default behavior is strict: if the requested start slot is occupied inside the configured occupancy radius, the add request is rejected. This matches the meeting requirement that adding a shuttle into an occupied start location should say no instead of silently placing another shuttle there.

Examples of valid add commands are:

```

ros2 topic pub --once /room_315/shuttle/add_cmd std_msgs/msg/String "{data: 'slot=3'}"
ros2 topic pub --once /room_315/shuttle/add_cmd std_msgs/msg/String "{data: 'entity=room315_shuttle_5
slot=4 speed=0.2'}"
ros2 topic pub --once /room_315/shuttle/add_cmd std_msgs/msg/String "{data: '{\"slot\":\"2\", \"entity
\": \"room315_shuttle_6\", \"speed\": \"0.15\"}'}"

```

Expected rejection examples are:

```

# Rejected if a shuttle is still inside the slot-1 occupancy radius.
ros2 topic pub --once /room_315/shuttle/add_cmd std_msgs/msg/String "{data: 'slot=1'}"

# Rejected because only slots 1, 2, 3, and 4 exist.
ros2 topic pub --once /room_315/shuttle/add_cmd std_msgs/msg/String "{data: 'slot=5'}"

```

7.14 Stopper command implementation

The stopper topic is `/room_315/stopper_states`. The command is intentionally simple: 0 means open or released, and 1 means closed or stop.

```

995         )
996
997     return {
998         'pose_scale_x': next_scale_x,
999         'pose_scale_y': next_scale_y,
1000         'pose_scale_origin_x': next_origin_x,
1001         'pose_scale_origin_y': next_origin_y,
1002         'pose_offset_x': next_x,
1003         'pose_offset_y': next_y,
1004         'pose_offset_z': next_z,
1005     }
1006
1007     def _on_stopper_command(self, message: String) -> None:
1008         try:
1009             updates = self._parse_stopper_command(message.data)
1010         except (ValueError, json.JSONDecodeError) as error:
1011             self.get_logger().error(str(error))
1012             return
1013
1014         self.stopper_states.update(updates)
1015         self.get_logger().info(f'Updated stopper states: {self.stopper_states}')
1016
1017     def _parse_stopper_command(self, raw_command: str) -> Dict[str, str]:
1018         assignments = self._parse_assignments(raw_command, 'Stopper')
1019         updates: Dict[str, str] = {}
1020         for raw_selector, raw_state in assignments:
1021             selector = raw_selector.strip().upper()
1022             state = self._normalize_stopper_state(raw_state)
1023             if selector == 'ALL':
1024                 for stopper_name in self.stopper_configs:
1025                     updates[stopper_name] = state
1026                 continue
1027             if selector not in self.stopper_configs:
1028                 allowed = ', '.join(['ALL', *sorted(self.stopper_configs)])
1029                 raise ValueError(
1030                     f'Unknown stopper selector {selector!r}; use one of: {allowed}.')
1031             updates[selector] = state
1032

```

Listing 7.15: Stopper command parser and accepted aliases.

The accepted stopper aliases are useful during manual testing:

- closed state: 1, ON, STOP, CLOSED, BLOCK, TRUE;
- open state: 0, OFF, OPEN, RELEASE, UNSTOP, FALSE;
- selector: A1, A2, A3, A4, or ALL.

Example commands:

```

ros2 topic pub --once /room_315/stopper_states std_msgs/msg/String "{data: 'A1=1'}"
ros2 topic pub --once /room_315/stopper_states std_msgs/msg/String "{data: 'A1=0'}"
ros2 topic pub --once /room_315/stopper_states std_msgs/msg/String "{data: 'A2=STOP A3=OPEN'}"
ros2 topic pub --once /room_315/stopper_states std_msgs/msg/String "{data: 'ALL=RELEASE'}"

```

7.15 Per-shuttle ON/OFF implementation

The shuttle ON/OFF topic is `/room_315/shuttle/control_cmd`. It controls movement permission for each shuttle independently. Turning a shuttle OFF does not delete it from Gazebo. It leaves the shuttle controlled by the node, sets the mode to `WAITING`, and marks `stopped_by` as `DISABLED`.

```

1034
1035 @staticmethod
1036 def _normalize_stopper_state(raw_state: str) -> str:
1037     state = str(raw_state).strip().upper()
1038     if state in {'1', 'ON', 'STOP', 'STOPPED', 'CLOSED', 'BLOCK', 'BLOCKED', 'TRUE'}:
1039         return '1'
1040     if state in {'0', 'OFF', 'OPEN', 'RELEASE', 'UNSTOP', 'UNBLOCK', 'FALSE'}:
1041         return '0'
1042     raise ValueError(
1043         f'Unknown stopper state {raw_state!r}; use 1/STOP/CLOSED or 0/OPEN/RELEASE.'
1044     )
1045
1046 def _on_shuttle_control_command(self, message: String) -> None:
1047     try:
1048         updates = self._parse_shuttle_control_command(message.data)
1049     except (ValueError, json.JSONDecodeError) as error:
1050         self.get_logger().error(str(error))
1051         return
1052
1053     for entity_name, enabled in updates.items():
1054         shuttle = self._find_shuttle(entity_name)
1055         if shuttle is None:
1056             self.get_logger().error(
1057                 f'Unknown shuttle {entity_name!r}; command ignored.'
1058             )
1059             continue
1060         shuttle.enabled = enabled
1061         if not enabled:
1062             shuttle.core.state.mode = WAITING
1063             shuttle.stopped_by = 'DISABLED'
1064             shuttle.stopper_distance_m = 0.0
1065         else:
1066             if shuttle.stopped_by == 'DISABLED':
1067                 shuttle.stopped_by = None
1068                 shuttle.stopper_distance_m = None
1069             if shuttle.core.state.mode == WAITING and shuttle.core.state.speed > 0.0:
1070                 shuttle.core.state.mode = MOVING
1071         self.get_logger().info(
1072             'Updated shuttle enable states: '
1073             f'{{ {shuttle.entity_name: shuttle.enabled for shuttle in self.shuttles} }}'
1074         )
1075
1076 def _parse_shuttle_control_command(self, raw_command: str) -> Dict[str, bool]:
1077     assignments = self._parse_assignments(raw_command, 'Shuttle control')
1078     keyed_payload = {key.strip().lower(): value for key, value in assignments}
1079     if {'entity', 'entity_name', 'name', 'shuttle'} & set(keyed_payload):
1080         entity_name = (
1081             keyed_payload.get('entity')
1082             or keyed_payload.get('entity_name')
1083             or keyed_payload.get('name')
1084             or keyed_payload.get('shuttle')
1085         )
1086         raw_state = (
1087             keyed_payload.get('enabled')
1088             or keyed_payload.get('state')
1089             or keyed_payload.get('mode')
1090             or keyed_payload.get('power')
1091         )
1092         if entity_name is None or raw_state is None:
1093             raise ValueError(
1094                 'Shuttle control command with entity=... must also include enabled=ON/OFF.'
1095             )
1096         return {entity_name: self._normalize_enabled_state(raw_state)}
1097
1098     updates: Dict[str, bool] = {}
1099     for raw_selector, raw_state in assignments:
1100         selector = raw_selector.strip()
1101         enabled = self._normalize_enabled_state(raw_state)
1102         if selector.upper() == 'ALL':
1103             for shuttle in self.shuttles:
1104                 updates[shuttle.entity_name] = enabled
1105     else:

```

```

1106         updates[selector] = enabled
1107     return updates

```

Listing 7.16: Per-shuttle ON/OFF control implementation.

Accepted ON aliases include 1, ON, ENABLE, START, RUN, and TRUE. Accepted OFF aliases include 0, OFF, DISABLE, STOP, PAUSE, and FALSE.

Example commands:

```

ros2 topic pub --once /room_315/shuttle/control_cmd std_msgs/msg/String "{data: 'room315_shuttle_2=OFF'}"
ros2 topic pub --once /room_315/shuttle/control_cmd std_msgs/msg/String "{data: 'room315_shuttle_2=ON'}"
ros2 topic pub --once /room_315/shuttle/control_cmd std_msgs/msg/String "{data: 'ALL=OFF'}"
ros2 topic pub --once /room_315/shuttle/control_cmd std_msgs/msg/String "{data: 'ALL=ON'}"
ros2 topic pub --once /room_315/shuttle/control_cmd std_msgs/msg/String "{data: '{\"entity\": \"room315_shuttle_3\", \"enabled\": \"OFF\"}'}"

```

7.16 Motion guard order

The ROS node wraps the pure kinematic core with runtime motion guards. The order is:

1. if the shuttle is disabled, keep it waiting;
2. if an active stopper is ahead, limit the step so the shuttle stops at the stop point;
3. apply collision avoidance against already occupied shuttle poses;
4. if neither guard blocks the motion, let the pure kinematic core advance.

```

1382         raw_poses.append(pose)
1383         gazebo_poses.append(gazebo_pose)
1384
1385         if pose.mode == FALLING:
1386             self.get_logger().error(
1387                 f'Shuttle {shuttle.entity_name} entered FALLING mode at '
1388                 f'segment={pose.current_segment}, s={pose.s:.3f}'
1389             )
1390
1391         self._publish_state(raw_poses, gazebo_poses)
1392         self._publish_sensor_state()
1393
1394     def _step_with_motion_guards(
1395         self,
1396         shuttle: ManagedShuttle,
1397         dt: float,
1398         occupied_poses: Dict[str, ShuttlePose],
1399     ) -> ShuttlePose:
1400         if not shuttle.enabled:
1401             shuttle.core.state.mode = WAITING
1402             shuttle.blocked_by = None
1403             shuttle.collision_distance_m = None
1404             shuttle.stopped_by = 'DISABLED'
1405             shuttle.stopper_distance_m = 0.0
1406             return shuttle.core.pose()
1407
1408         shuttle.stopped_by = None
1409         shuttle.stopper_distance_m = None
1410         active_stop = self._active_stopper_ahead(shuttle)
1411         effective_dt = dt
1412         stop_reached = False
1413         if active_stop is not None and shuttle.core.state.speed > 0.0:
1414             stopper_name, stop_point, distance_m = active_stop
1415             if distance_m <= 1e-6:
1416                 shuttle.core.state.s = stop_point.stop_s
1417                 shuttle.core.state.mode = WAITING
1418                 shuttle.blocked_by = None
1419                 shuttle.collision_distance_m = None
1420                 shuttle.stopped_by = stopper_name
1421                 shuttle.stopper_distance_m = 0.0
1422                 return shuttle.core.pose()
1423
1424             time_to_stop = distance_m / shuttle.core.state.speed
1425             if time_to_stop <= dt:
1426                 effective_dt = time_to_stop
1427                 stop_reached = True
1428
1429         pose = self._step_with_collision_avoidance(
1430             shuttle=shuttle,
1431             dt=effective_dt,
1432             occupied_poses=occupied_poses,
1433         )
1434         if shuttle.blocked_by is not None or pose.mode == FALLING:
1435             return pose
1436
1437         if stop_reached and active_stop is not None:
1438             stopper_name, stop_point, _distance_m = active_stop
1439             if shuttle.core.state.current_segment == stop_point.segment:
1440                 shuttle.core.state.s = stop_point.stop_s
1441                 shuttle.core.state.mode = WAITING
1442                 shuttle.stopped_by = stopper_name
1443                 shuttle.stopper_distance_m = 0.0
1444                 return shuttle.core.pose()
1445
1446         shuttle.stopped_by = None
1447         shuttle.stopper_distance_m = None
1448         return pose
1449
1450     def _active_stopper_ahead(
1451         self,
1452         shuttle: ManagedShuttle,
1453     ) -> tuple[str, StopPoint, float] | None:

```

```

1454     state = shuttle.core.state
1455     candidates: list[tuple[str, StopPoint, float]] = []

```

Listing 7.17: Motion guard wrapper for disabled shuttles and active stoppers.

This order matters. A disabled shuttle should not move even if the path is clear. A closed stopper should stop a shuttle before the switch. Collision avoidance then prevents two active shuttles from merging into each other.

7.17 Collision avoidance behavior

Collision avoidance uses center-distance checks between the Gazebo-space poses of shuttles. The default threshold is 0.33 m. If the proposed full time step would collide, the code performs a binary search over the time step and moves only to the last safe position, then enters WAITING.

```

1457         if self.stopper_states.get(stopper_name, '0') != '1':
1458             continue
1459         for stop_point in stopper_config.stop_points:
1460             if stop_point.segment != state.current_segment:
1461                 continue
1462             distance_m = stop_point.stop_s - state.s
1463             if distance_m >= -1e-6:
1464                 candidates.append((stopper_name, stop_point, max(0.0, distance_m)))
1465         if not candidates:
1466             return None
1467         return sorted(candidates, key=lambda item: item[2])[0]
1468
1469     def _step_with_collision_avoidance(
1470         self,
1471         shuttle: ManagedShuttle,
1472         dt: float,
1473         occupied_poses: Dict[str, ShuttlePose],
1474     ) -> ShuttlePose:
1475         if not self.enable_collision_avoidance or dt <= 0.0:
1476             shuttle.blocked_by = None
1477             shuttle.collusion_distance_m = None
1478             return shuttle.core.step(dt, switch_states=self.switch_states)
1479
1480         start_state = self._snapshot_shuttle_state(shuttle)
1481         start_pose = shuttle.core.pose()
1482         start_gazebo_pose = self._to_gazebo_pose(start_pose)
1483         blockers = self._collision_blockers(shuttle.entity_name, start_gazebo_pose, occupied_poses)
1484         if blockers:
1485             blocker_name, distance = blockers[0]
1486             self._restore_shuttle_state(shuttle, start_state)
1487             shuttle.core.state.mode = WAITING
1488             shuttle.blocked_by = blocker_name
1489             shuttle.collusion_distance_m = distance
1490             return shuttle.core.pose()
1491
1492         self._restore_shuttle_state(shuttle, start_state)
1493         proposed_pose = shuttle.core.step(dt, switch_states=self.switch_states)
1494         proposed_gazebo_pose = self._to_gazebo_pose(proposed_pose)
1495         blockers = self._collision_blockers(
1496             shuttle.entity_name,
1497             proposed_gazebo_pose,
1498             occupied_poses,
1499         )
1500         if not blockers:
1501             shuttle.blocked_by = None
1502             shuttle.collusion_distance_m = None
1503             return proposed_pose
1504
1505         high = dt
1506         low = 0.0
1507         blocker_name, distance = blockers[0]

```

```

1508         for _ in range(max(1, self.collision_search_iterations)):
1509             mid = 0.5 * (low + high)
1510             self._restore_shuttle_state(shuttle, start_state)
1511             mid_pose = shuttle.core.step(mid, switch_states=self.switch_states)
1512             mid_gazebo_pose = self._to_gazebo_pose(mid_pose)
1513             mid_blockers = self._collision_blockers(
1514                 shuttle.entity_name,
1515                 mid_gazebo_pose,
1516                 occupied_poses,
1517             )

```

Listing 7.18: Collision avoidance with binary search for the last safe step.

This is a kinematic approximation, not full contact physics. It is intentionally simple for the current phase. Its purpose is to prevent visual overlap and to model the practical rule: if a shuttle reaches another shuttle, it waits instead of passing through it.

7.18 Sensor event publication

The sensor interface is educational and workflow-oriented. It publishes approach events whenever a shuttle is within the configured sensor distance before a stopper.

```

1648         'Gazebo set_pose service is not ready yet; still publishing pose topic.'
1649     )
1650     shuttle.set_pose_warning_logged = True
1651     return
1652
1653     request = SetEntityPose.Request()
1654     request.entity.name = shuttle.entity_name
1655     request.entity.type = Entity.MODEL
1656     request.pose = pose_message.pose
1657     shuttle.pending_set_pose = self.set_pose_client.call_async(request)
1658     shuttle.last_gazebo_set_pose_time = now
1659
1660     def _sensor_events(self) -> list[dict]:
1661         events = []
1662         for shuttle in self.shuttles:
1663             state = shuttle.core.state
1664             for stopper_name, stopper_config in self.stopper_configs.items():
1665                 for stop_point in stopper_config.stop_points:
1666                     if stop_point.segment != state.current_segment:
1667                         continue
1668                     distance_m = stop_point.stop_s - state.s
1669                     if 0.0 <= distance_m <= stop_point.sensor_distance_m:
1670                         events.append(
1671                             {
1672                                 'sensor': f'{stopper_name}_APPROACH',
1673                                 'stopper': stopper_name,
1674                                 'before_switch': stopper_config.before_switch,
1675                                 'entity_name': shuttle.entity_name,
1676                                 'segment': state.current_segment,
1677                                 'distance_m': distance_m,
1678                                 'stopper_state': self.stopper_states.get(stopper_name, '0'),
1679                                 'workflow': 'sensor -> stop shuttle -> move switch -> unstop shuttle',
1680                             }
1681                         )

```

Listing 7.19: Switch-approach sensor event generation.

A sensor event contains:

- **sensor**: for example `A3_APPROACH`;
- **stopper**: the stopper that can stop the shuttle;
- **before_switch**: the protected switch;

- `entity_name`: which shuttle triggered the event;
- `segment`: which segment the shuttle is currently on;
- `distance_m`: remaining distance to the stop point;
- `stopper_state`: whether the stopper is currently open or closed;
- `workflow`: the intended sequence, `sensor -> stop shuttle -> move switch -> unstop shuttle`.

When four shuttles are running, the sensor topic can contain events from any of them. The event is not anonymous. The `entity_name` field tells exactly which shuttle the event belongs to.

7.19 Published state payload

The state topic is `/room_315/shuttle/state`. It publishes a JSON string with global switch and stopper states plus a list of shuttles. Each shuttle entry includes the raw rail pose, transformed Gazebo pose, segment, arc length, mode, enabled flag, blocker name, stopper name, start slot, and speed.

```

1683
1684     def _publish_sensor_state(self) -> None:
1685         message = String()
1686         message.data = json.dumps(
1687             {
1688                 'sensors': self._sensor_events(),
1689                 'stopper_states': self.stopper_states,
1690             },
1691             sort_keys=True,
1692         )
1693         self.sensor_state_publisher.publish(message)
1694
1695     def _publish_state(
1696         self,
1697         raw_poses: list[ShuttlePose],
1698         gazebo_poses: list[ShuttlePose],
1699     ) -> None:
1700         shuttles_payload = []
1701         for shuttle, pose, gazebo_pose in zip(self.shuttles, raw_poses, gazebo_poses):
1702             shuttles_payload.append(
1703                 {
1704                     **asdict(pose),
1705                     'entity_name': shuttle.entity_name,

```

Listing 7.20: Per-shuttle state fields published on the state topic.

This topic is the best first debug topic because it tells whether a shuttle is stopped by an OFF command, stopped by a stopper, blocked by another shuttle, or in **FALLING**.

Chapter 8

Complete Feature Usage Manual

This chapter lists the practical commands for every current feature. The commands are intentionally copy-and-paste friendly.

8.1 Terminal setup used by all examples

Every terminal should start with:

```
cd /home/tiago/ALI_ros2_ws
source /opt/ros/jazzy/setup.bash
source install/setup.bash
```

If the workspace was modified, rebuild first:

```
cd /home/tiago/ALI_ros2_ws
source /opt/ros/jazzy/setup.bash

colcon build --base-paths \
  src/mfja_3rd_floor_gz/mfja_robot_control_config \
  src/mfja_3rd_floor_gz/mfja_3rd_floor_description \
  src/mfja_3rd_floor_gz/mfja_room_315_bringup \
  src/mfja_3rd_floor_gz/mfja_3rd_floor_bringup \
  src/mfja_3rd_floor_gz/mfja_3rd_floor_gz \
  --packages-select \
  mfja_robot_control_config \
  mfja_3rd_floor_description \
  mfja_room_315_bringup \
  mfja_3rd_floor_bringup \
  mfja_3rd_floor_gz \
  --symlink-install

source install/setup.bash
```

8.2 Complete feature checklist

The current repository contains both the older multi-robot simulation work and the newer Room 315 kinematic shuttle work. The table below maps each feature to the practical section that explains how to run it.

Feature	Where it is operated from
Build and source the workspace	Terminal setup section in this chapter.

Feature	Where it is operated from
Launch Room 315 only	<code>mfja_room_315_bringup room_315_only.launch.py.</code>
Launch the full third floor	<code>mfja_3rd_floor_bringup full_floor.launch.py.</code>
Spawn no robots, selected robots, or all robots	The <code>robots:=...</code> launch argument.
Control industrial robot arms	Namespaced <code>/robot_name/joint_trajectory</code> topics.
Control the TIAGo mobile base	The <code>/tiago1/cmd_vel</code> topic.
Run synchronized robot motions	<code>multi_robot_sync_demo.py.</code>
Run one or more Room 315 shuttles	<code>room_315_kinematic_shuttle_node.py.</code>
Add shuttles during simulation	<code>/room_315/shuttle/add_cmd.</code>
Turn shuttles ON or OFF	<code>/room_315/shuttle/control_cmd.</code>
Change route switches	<code>/room_315/switch_states.</code>
Stop and release shuttles before switches	<code>/room_315/stopper_states.</code>
Read educational sensor events	<code>/room_315/sensors/switch_approach.</code>
Inspect shuttle state and collision holds	<code>/room_315/shuttle/state.</code>
Validate CSV paths and the rail network	<code>room_315_csv_preprocessor.py,</code> <code>room_315_network_validator.py,</code> and <code>room_315_segment_plot.py.</code>

8.3 Terminal 1: run Room 315 only

Use this when testing Room 315 in isolation:

```
cd /home/tiago/ALI_ros2_ws
source /opt/ros/jazzy/setup.bash
source install/setup.bash

ros2 launch mfja_room_315_bringup room_315_only.launch.py \
  robots:=none \
  start_paused:=false
```

If Gazebo opens paused, press the play button or restart with `start_paused:=false`.

8.4 Terminal 1 alternative: run the full floor

Use this when testing Room 315 inside the complete third-floor world:

```
cd /home/tiago/ALI_ros2_ws
```

```
source /opt/ros/jazzy/setup.bash
source install/setup.bash

ros2 launch mfja_3rd_floor_bringup full_floor.launch.py \
  robots:=none \
  start_paused:=false
```

The shuttle node must use the same Gazebo world name as the running world. For Room-only, use `room_315_only`. For the full floor, use `mfja_3rd_floor`.

8.5 Terminal 1 robot launch options

The launch argument `robots:=...` controls which robot models are spawned with the world. It supports:

- `robots:=none`: start the world without robots, useful for shuttle-only testing;
- no `robots` argument or `robots:=`: use the YAML enabled flags;
- `robots:=all`: spawn every robot entry from the selected YAML file;
- full names such as `kuka1` and `tiago1`;
- shortcuts such as `kuka`, `staubli`, `hc10`, `hc10dt`, and `tiago`;
- numeric indices by YAML order such as `1,2,5`.

Room 315 with all configured robot entries:

```
cd /home/tiago/ALI_ros2_ws
source /opt/ros/jazzy/setup.bash
source install/setup.bash

ros2 launch mfja_room_315_bringup room_315_only.launch.py \
  robots:=all \
  start_paused:=false
```

Room 315 with selected robots only:

```
cd /home/tiago/ALI_ros2_ws
source /opt/ros/jazzy/setup.bash
source install/setup.bash

ros2 launch mfja_room_315_bringup room_315_only.launch.py \
  robots:=kuka1,staubli1,yaskawa_hc10_1 \
  start_paused:=false
```

Full floor with all enabled robots from `robots.yaml`:

```
cd /home/tiago/ALI_ros2_ws
source /opt/ros/jazzy/setup.bash
source install/setup.bash

ros2 launch mfja_3rd_floor_bringup full_floor.launch.py \
  start_paused:=false
```

Full floor with all configured robots, even if a YAML entry is disabled:

```
cd /home/tiago/ALI_ros2_ws
source /opt/ros/jazzy/setup.bash
source install/setup.bash

ros2 launch mfja_3rd_floor_bringup full_floor.launch.py \
  robots:=all \
  start_paused:=false
```

Full floor with selected robots by shortcuts:

```
cd /home/tiago/ALI_ros2_ws
source /opt/ros/jazzy/setup.bash
source install/setup.bash

ros2 launch mfja_3rd_floor_bringup full_floor.launch.py \
  robots:=kuka,staubli,hc10,hc10dt,tiago \
  start_paused:=false
```

8.6 Robot terminal A: verify robot topics

Open a new terminal after the world is running:

```
cd /home/tiago/ALI_ros2_ws
source /opt/ros/jazzy/setup.bash
source install/setup.bash

ros2 topic list | grep -E '^(kuka1|staubli1|yaskawa_hc10_1|yaskawa_hc10dt_1|tiago1)/'

ros2 topic info /kuka1/joint_trajectory
ros2 topic info /staubli1/joint_trajectory
ros2 topic info /yaskawa_hc10_1/joint_trajectory
ros2 topic info /yaskawa_hc10dt_1/joint_trajectory
ros2 topic info /tiago1/joint_trajectory
ros2 topic info /tiago1/cmd_vel
```

The joint trajectory topics are ROS-to-Gazebo command topics. The joint state, odometry, and progress topics can be inspected while the robots move:

```
ros2 topic echo /kuka1/joint_states
ros2 topic echo /staubli1/joint_states
ros2 topic echo /yaskawa_hc10_1/joint_states
ros2 topic echo /yaskawa_hc10dt_1/joint_states
ros2 topic echo /tiago1/odom
```

8.7 Robot terminal B: command individual robot arms

KUKA KR6 R900 sixx:

```
cd /home/tiago/ALI_ros2_ws
source /opt/ros/jazzy/setup.bash
source install/setup.bash

ros2 topic pub --once /kuka1/joint_trajectory \
trajectory_msgs/msg/JointTrajectory \
"{joint_names: ['joint_a1','joint_a2','joint_a3','joint_a4','joint_a5','joint_a6'], points: [{positions:
  [0.6,-1.0,1.1,0.0,0.6,0.0], time_from_start: {sec: 3, nanosec: 0}}]}"
```

Staubli TX2-60L:

```
cd /home/tiago/ALI_ros2_ws
source /opt/ros/jazzy/setup.bash
source install/setup.bash

ros2 topic pub --once /staubli1/joint_trajectory \
trajectory_msgs/msg/JointTrajectory \
"{joint_names: ['joint_1','joint_2','joint_3','joint_4','joint_5','joint_6'], points: [{positions:
  [0.1,0.4,-0.6,0.0,0.5,0.0], time_from_start: {sec: 3, nanosec: 0}}]}"
```

Yaskawa HC10:

```
cd /home/tiago/ALI_ros2_ws
source /opt/ros/jazzy/setup.bash
source install/setup.bash

ros2 topic pub --once /yaskawa_hc10_1/joint_trajectory \
trajectory_msgs/msg/JointTrajectory \
"{joint_names: ['joint_1_s','joint_2_l','joint_3_u','joint_4_r','joint_5_b','joint_6_t'], points: [{
  positions: [0.0,-0.6,0.8,0.0,0.5,0.0], time_from_start: {sec: 3, nanosec: 0}}]}"
```

Yaskawa HC10DT:

```
cd /home/tiago/ALI_ros2_ws
source /opt/ros/jazzy/setup.bash
source install/setup.bash

ros2 topic pub --once /yaskawa_hc10dt_1/joint_trajectory \
trajectory_msgs/msg/JointTrajectory \
"{joint_names: ['joint_1_s','joint_2_l','joint_3_u','joint_4_r','joint_5_b','joint_6_t'], points: [{
  positions: [0.0,-0.5,0.7,0.0,0.4,0.0], time_from_start: {sec: 3, nanosec: 0}}]}"
```

TIAGo arm and head:

```
cd /home/tiago/ALI_ros2_ws
source /opt/ros/jazzy/setup.bash
source install/setup.bash

ros2 topic pub --once /tiago1/joint_trajectory \
trajectory_msgs/msg/JointTrajectory \
"{joint_names: ['torso_lift_joint','arm_1_joint','arm_2_joint','arm_3_joint','arm_4_joint','arm_5_joint',
  'arm_6_joint','arm_7_joint','head_1_joint','head_2_joint'], points: [{positions:
  [0.10,0.3,-0.5,-0.4,1.0,0.2,-0.2,0.1,0.2,-0.2], time_from_start: {sec: 4, nanosec: 0}}]}"
```

8.8 Robot terminal C: command the TIAGo base

Move the TIAGo base slowly:

```
cd /home/tiago/ALI_ros2_ws
source /opt/ros/jazzy/setup.bash
source install/setup.bash

ros2 topic pub -r 20 /tiago1/cmd_vel geometry_msgs/msg/Twist \
"{linear: {x: 0.15, y: 0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 0.20}}"
```

Stop the TIAGo base from another terminal, or press Ctrl-C in the publishing terminal and send one zero command:

```
cd /home/tiago/ALI_ros2_ws
source /opt/ros/jazzy/setup.bash
source install/setup.bash

ros2 topic pub --once /tiago1/cmd_vel geometry_msgs/msg/Twist \
"{linear: {x: 0.0, y: 0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 0.0}}"
```

8.9 Robot terminal D: synchronized multi-robot commands

List supported joints and default poses:

```
cd /home/tiago/ALI_ros2_ws
source /opt/ros/jazzy/setup.bash
source install/setup.bash

ros2 run mfja_robot_control_config multi_robot_sync_demo.py --list-joints
```

Move every enabled robot to its default preset:

```
cd /home/tiago/ALI_ros2_ws
source /opt/ros/jazzy/setup.bash
source install/setup.bash

ros2 run mfja_robot_control_config multi_robot_sync_demo.py \
--trajectory-duration 4.0
```

Move only selected robots with explicit goals:

```
cd /home/tiago/ALI_ros2_ws
source /opt/ros/jazzy/setup.bash
source install/setup.bash

ros2 run mfja_robot_control_config multi_robot_sync_demo.py \
--goal kuka1=1.2,-1.2,1.4,0.0,0.3,0.0 \
--goal Staubli1=0.1,0.4,-0.6,0.0,0.5,0.0 \
--trajectory-duration 4.0
```

Move TIAGo arm and base together for a short demo:

```
cd /home/tiago/ALI_ros2_ws
source /opt/ros/jazzy/setup.bash
source install/setup.bash

ros2 run mfja_robot_control_config multi_robot_sync_demo.py \
--goal tiago1=0.10,0.3,-0.5,-0.4,1.0,0.2,-0.2,0.1,0.2,-0.2 \
--tiago-base-linear 0.10 \
--tiago-base-angular 0.15 \
--tiago-base-duration 3.0 \
--trajectory-duration 4.0
```

8.10 Terminal 2: bridge the Gazebo set-pose service

For Room-only:

```
cd /home/tiago/ALI_ros2_ws
source /opt/ros/jazzy/setup.bash
source install/setup.bash
export GZ_PARTITION=room315

ros2 run ros_gz_bridge parameter_bridge \
/world/room_315_only/set_pose@ros_gz_interfaces/srv/SetEntityPose
```

For the full floor:

```
cd /home/tiago/ALI_ros2_ws
source /opt/ros/jazzy/setup.bash
source install/setup.bash

ros2 run ros_gz_bridge parameter_bridge \
/world/mfja_3rd_floor/set_pose@ros_gz_interfaces/srv/SetEntityPose
```

Check that the service exists:

```
ros2 service list | grep set_pose
```

8.11 Terminal 3: run one shuttle

Room-only one-shuttle command:

```
cd /home/tiago/ALI_ros2_ws
source /opt/ros/jazzy/setup.bash
source install/setup.bash
```

```
ros2 run mfja_robot_control_config room_315_kinematic_shuttle_node.py --ros-args \
  -p gazebo_world_name:=room_315_only \
  -p start_slot:=2 \
  -p enable_gazebo_set_pose:=true \
  -p gazebo_entity_name:=room315_shuttle_1 \
  -p speed:=0.2 \
  -p gazebo_set_pose_rate_hz:=10.0
```

Full-floor one-shuttle command:

```
cd /home/tiago/ALI_ros2_ws
source /opt/ros/jazzy/setup.bash
source install/setup.bash

ros2 run mfja_robot_control_config room_315_kinematic_shuttle_node.py --ros-args \
  -p gazebo_world_name:=mfja_3rd_floor \
  -p start_slot:=2 \
  -p enable_gazebo_set_pose:=true \
  -p gazebo_entity_name:=room315_shuttle_1 \
  -p speed:=0.2 \
  -p gazebo_set_pose_rate_hz:=10.0
```

8.12 Terminal 3: four-shuttle startup

```
cd /home/tiago/ALI_ros2_ws
source /opt/ros/jazzy/setup.bash
source install/setup.bash

ros2 run mfja_robot_control_config room_315_kinematic_shuttle_node.py --ros-args \
  -p gazebo_world_name:=room_315_only \
  -p shuttle_count:=4 \
  -p start_slots:=1,2,3,4 \
  -p gazebo_entity_names:=room315_shuttle_1,room315_shuttle_2,room315_shuttle_3,room315_shuttle_4 \
  -p enable_gazebo_set_pose:=true \
  -p speed:=0.2 \
  -p gazebo_set_pose_rate_hz:=10.0
```

If running in the full-floor world, change only:

```
-p gazebo_world_name:=mfja_3rd_floor
```

8.13 Terminal 4: inspect state continuously

```
cd /home/tiago/ALI_ros2_ws
source /opt/ros/jazzy/setup.bash
source install/setup.bash

ros2 topic echo /room_315/shuttle/state
```

Useful fields:

- `entity_name`: identifies which shuttle the row belongs to;
- `current_segment`: current rail segment;
- `s`: arc-length progress on that segment;
- `mode`: MOVING, WAITING, or FALLING;
- `enabled`: per-shuttle ON/OFF state;

- `stopped_by`: stopper name or DISABLED;
- `blocked_by`: another shuttle if collision avoidance is holding it;
- `start_slot`: original start slot used for that shuttle.

8.14 Terminal 5: inspect sensor events

```
cd /home/tiago/ALI_ros2_ws
source /opt/ros/jazzy/setup.bash
source install/setup.bash

ros2 topic echo /room_315/sensors/switch_approach
```

Example event:

```
data: '{"sensors": [{"before_switch": "A3", "distance_m": 0.214, "entity_name": "room315_shuttle_4", "segment": "A23", "sensor": "A3_APPROACH", "stopper": "A3", "stopper_state": "0", "workflow": "sensor -> stop shuttle -> move switch -> unstop shuttle"}], "stopper_states": {"A1": "0", "A2": "0", "A3": "0", "A4": "0"}}'
```

This means shuttle `room315_shuttle_4` is approaching switch `A3` on segment `A23`. The remaining distance to the `A3` stop point is about 0.214 m. The stopper is currently open because `stopper_state` is 0.

8.15 Switch commands

Switches are controlled through:

```
/room_315/switch_states
```

Use these commands to test all switches:

```
ros2 topic pub --once /room_315/switch_states std_msgs/msg/String "{data: 'A1=G'}"
ros2 topic pub --once /room_315/switch_states std_msgs/msg/String "{data: 'A1=S'}"
ros2 topic pub --once /room_315/switch_states std_msgs/msg/String "{data: 'A2=G'}"
ros2 topic pub --once /room_315/switch_states std_msgs/msg/String "{data: 'A2=S'}"
ros2 topic pub --once /room_315/switch_states std_msgs/msg/String "{data: 'A3=G'}"
ros2 topic pub --once /room_315/switch_states std_msgs/msg/String "{data: 'A3=S'}"
ros2 topic pub --once /room_315/switch_states std_msgs/msg/String "{data: 'A4=G'}"
ros2 topic pub --once /room_315/switch_states std_msgs/msg/String "{data: 'A4=S'}"
```

Set all switches at once:

```
ros2 topic pub --once /room_315/switch_states std_msgs/msg/String "{data: 'ALL=G'}"
ros2 topic pub --once /room_315/switch_states std_msgs/msg/String "{data: 'ALL=S'}"
```

Set a mixed route in one command:

```
ros2 topic pub --once /room_315/switch_states std_msgs/msg/String "{data: 'A1=S A2=G A3=S A4=G'}"
```

The switch command topic updates both routing logic and visual switch commands. Do not command only `/mfja/conveyor/switch_cmd` if the shuttle route must change. That topic is visual-only from the routing point of view.

8.16 Stop-switch-release workflow

The intended teaching workflow is:

1. watch /room_315/sensors/switch_approach;
2. close the stopper before the switch;
3. wait until the shuttle is stopped;
4. move the switch;
5. open the stopper.

Example for switch A3:

```
# Stop before A3.
ros2 topic pub --once /room_315/stopper_states std_msgs/msg/String "{data: 'A3=1'}"

# Move the switch while the shuttle is stopped.
ros2 topic pub --once /room_315/switch_states std_msgs/msg/String "{data: 'A3=S'}"

# Release the shuttle.
ros2 topic pub --once /room_315/stopper_states std_msgs/msg/String "{data: 'A3=0'}"
```

The same pattern applies to A1, A2, and A4.

8.17 Add shuttles during simulation

After the node is already running, add a shuttle from a free slot:

```
ros2 topic pub --once /room_315/shuttle/add_cmd std_msgs/msg/String "{data: 'slot=3'}"
```

Add with a custom entity name and speed:

```
ros2 topic pub --once /room_315/shuttle/add_cmd std_msgs/msg/String "{data: 'entity=room315_shuttle_5
slot=4 speed=0.2'}"
```

If the requested slot is still occupied, the node rejects the command. This is expected behavior. Wait until the current shuttle has moved away from the start slot, or choose another free slot.

8.18 Turn individual shuttles ON or OFF

Turn one shuttle OFF:

```
ros2 topic pub --once /room_315/shuttle/control_cmd std_msgs/msg/String "{data: 'room315_shuttle_2=OFF
'}"
```

Turn it ON again:

```
ros2 topic pub --once /room_315/shuttle/control_cmd std_msgs/msg/String "{data: 'room315_shuttle_2=ON'}"
```

Turn all shuttles OFF or ON:

```
ros2 topic pub --once /room_315/shuttle/control_cmd std_msgs/msg/String "{data: 'ALL=OFF'}"
ros2 topic pub --once /room_315/shuttle/control_cmd std_msgs/msg/String "{data: 'ALL=ON'}"
```

The state topic should show `enabled=false` and `stopped_by=DISABLED` for a disabled shuttle.

8.19 Pose calibration controls

The accepted CSV coordinates already include the calibration that was found during testing. Normal operation should use default scale and offset values. However, runtime calibration is still available for future alignment tests.

Start with temporary X and Y correction:

```
ros2 run mfja_robot_control_config room_315_kinematic_shuttle_node.py --ros-args \
-p gazebo_world_name:=room_315_only \
-p enable_gazebo_set_pose:=true \
-p gazebo_entity_name:=room315_shuttle_1 \
-p speed:=0.2 \
-p gazebo_set_pose_rate_hz:=10.0 \
-p pose_scale_x:=1.0 \
-p pose_scale_y:=1.0 \
-p pose_offset_x:=0.0 \
-p pose_offset_y:=0.0 \
-p pose_offset_z:=0.0
```

Change offset while the node is running:

```
ros2 topic pub --once /room_315/shuttle/pose_offset_cmd std_msgs/msg/String "{data: 'x=-0.02 y=0.01 z=0.00'}"
```

This should be used only for calibration experiments. Once the correct values are known, the preferred method is to bake the correction into the CSV path data and run the node with default calibration parameters.

8.20 Validation commands

Run preprocessing and validation:

```
cd /home/tiago/ALI_ros2_ws
source /opt/ros/jazzy/setup.bash
source install/setup.bash

ros2 run mfja_robot_control_config room_315_csv_preprocessor.py
ros2 run mfja_robot_control_config room_315_network_validator.py
ros2 run mfja_robot_control_config room_315_segment_plot.py
```

Expected validator result:

```
Status: PASS (0 warnings)
```

For the continuous path validator, the expected result is:

```
Validated continuous paths for 14 segments.
Status: PASS (0 warnings)
```

Generated files:

- normalized CSV output: mfja_robot_control_config/config/room_315_kinematics/normalized_segments;
- segment summary: mfja_robot_control_config/config/room_315_kinematics/segment_summary.yaml;
- validation report: mfja_robot_control_config/config/room_315_kinematics/validation_report.yaml;
- continuous path report: mfja_robot_control_config/config/room_315_kinematics/continuous_path_report.yaml;

- network plot: `mfja_robot_control_config/config/room_315_kinematics/debug_plots/network_validation.png`.
- continuous path comparison plot: `mfja_robot_control_config/config/room_315_kinematics/debug_plots/continuous_path_validation.png`.

8.21 Path backend selection

Normal demos should use the continuous path backend:

```
-p path_backend:=cubic_hermite
```

The old CSV polyline backend is still available for comparison and debugging:

```
-p path_backend:=polyline
```

The recommended policy is to run with `cubic_hermite` and keep `polyline` as a reference. The validator compares these two backends, so removing `polyline` would remove the easiest way to prove that the continuous path did not drift away from the measured CSV data.

8.22 Troubleshooting table

Symptom	Most likely cause and action
Gazebo <code>set_pose</code> service is not ready yet	The bridge is missing, the world name is wrong, or Gazebo is not fully ready. Run <code>ros2 service list grep set_pose</code> and make sure the shuttle node uses the same <code>gazebo_world_name</code> .
Shuttle does not appear in the full-floor world	The node may still be using <code>room_315_only</code> . Start it with <code>-p gazebo_world_name:=mfja_3rd_floor</code> .
Switch moves visually but the route does not change	The command was likely sent to the visual topic only. Send route commands to <code>/room_315/switch_states</code> .
Shuttle stops at a switch approach	Check <code>stopped_by</code> on the state topic. If it is a stopper name, release that stopper with <code>/room_315/stopper_states</code> .
Shuttle is waiting with <code>stopped_by=DISABLED</code>	It was turned OFF by <code>/room_315/shuttle/control_cmd</code> . Send <code>room315_shuttle_N=ON</code> .
Shuttle is waiting with <code>blocked_by</code> set	Collision avoidance is active. The named shuttle is too close. Move, stop, or release shuttles so spacing becomes safe.
Add command is rejected because a slot is occupied	This is the current correct behavior. Wait until the slot is clear or choose another slot.

Symptom	Most likely cause and action
Add command for slot 5 is rejected	Only four start slots exist in the current accepted configuration: 1, 2, 3, and 4.
Shuttle enters FALLING	The current segment has no valid successor under the current switch state. Check the routing table and current switch states.

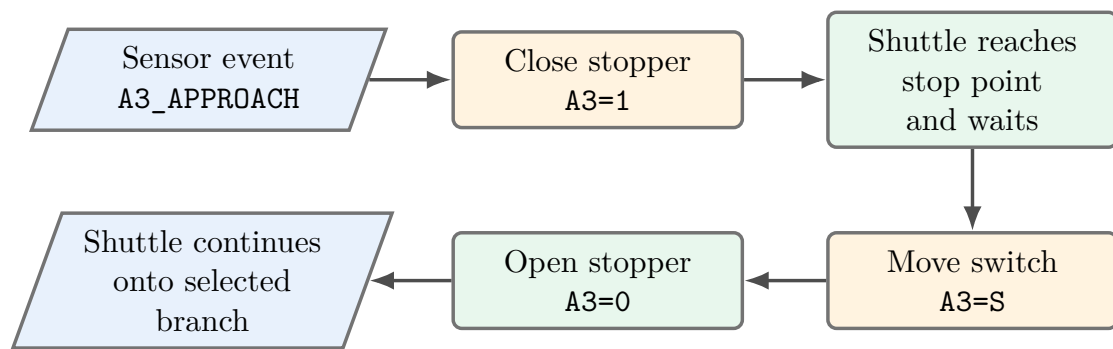


Figure 8.1: Stopper-based switch workflow. The stopper and switch are separate primitives: the stopper creates a safe waiting point, and the switch selects the future route.

Chapter 9

What Was Built in This Phase

This chapter summarizes the concrete engineering changes that define the current version of the repository.

9.1 Repository cleanup and package separation

The repository was cleaned into a meta-repository shape. Root-level package artifacts were removed or moved into the packages that own them. The current structure separates:

- Gazebo descriptions and worlds;
- launch entry points for Room 315 and the full floor;
- shared robot and shuttle control configuration;
- compatibility launch entry points;
- repository-level documentation.

This makes the project easier to build with explicit `colcon` package selection and prevents unrelated assets from being mixed at the root.

9.2 Dynamic shuttle work removed from the current main path

Earlier dynamic/contact-based shuttle experiments were removed from this branch because they were not the accepted direction for the current phase. The current approach is kinematic-first:

- validate CSV rail geometry offline;
- validate explicit topology and switch routing;
- run one-shuttle logic without Gazebo;
- add ROS and Gazebo pose control;
- add multiple shuttles, stoppers, sensors, and simple collision avoidance;

- postpone contact dynamics, wheels, and rail contact physics.

This order reduces risk because path logic can be tested before physics complexity is introduced.

9.3 Room 315 rail graph and validation pipeline

The rail network was split into directed CSV segments. The accepted segment names are:

```
A1G A1S A2G A2S A3G A3S A4G A4S A12E A12I A14 A23 A34E A34I
```

The validation tools produce:

- normalized CSV files;
- segment lengths;
- duplicate-removal counts;
- endpoint-to-anchor distances;
- gap warnings above 0.05 m;
- tangent mismatch warnings above 25 deg;
- a debug plot with node labels and segment labels.

The latest expected result is a clean pass with no warnings.

9.4 Continuous path backend added

The shuttle geometry layer now supports parametrized path backends. The current recommended runtime backend is `cubic_hermite`. It turns calibrated CSV points into a continuous arc-length path so the shuttle pose and yaw change more smoothly through curves.

The original `polyline` backend remains available on purpose. It is the reference backend used to compare against the measured CSV data. Keeping both backends gives a safe workflow:

```
normal demos:      path_backend:=cubic_hermite
debug/reference use: path_backend:=polyline
```

The new validator produces `mfja_robot_control_config/config/room_315_kinematics/continuous_path_report.yaml` and `mfja_robot_control_config/config/room_315_kinematics/debug_plots/continuous_path_validation.png`. The current result is **PASS** (0 warnings), with millimeter-level deviation from the CSV polyline reference.

9.5 Switches made independent and visible

The shuttle route switch state is controlled by `/room_315/switch_states`. Commands sent there update both the graph routing state and the visual conveyor switch command topic. This gives one operator command for logical and visual behavior.

The current logical switch states are **G** and **S**, matching the segment suffixes in the network file. Historical aliases such as left/right or specific visual names may exist in the visual layer, but the current route graph uses **G/S** as the stable internal representation.

9.6 Stopper primitive added before switches

Independent binary stoppers were added before switches **A1**, **A2**, **A3**, and **A4**. This fills the workflow gap from the meeting requirement:

```
sensor -> stop shuttle -> move switch -> unstop shuttle
```

The stopper primitive is intentionally separate from the switch primitive. A stopper answers the question: should the shuttle wait before entering this switch area? A switch answers the question: which route is selected?

9.7 Sensor workflow added

The sensor topic publishes approach events before a switch. This is not a full hardware sensor simulation yet. It is a deterministic software sensor derived from the shuttle arc length and the configured stop point. It is enough for the teaching and testing workflow because it tells the operator:

- which shuttle is approaching;
- which switch is ahead;
- which stopper protects that switch;
- how far the shuttle is from the stop point;
- whether the stopper is currently open or closed.

9.8 Per-shuttle control added

Each shuttle can now be turned ON or OFF independently. This was added because the system needs an operational primitive that can pause a specific shuttle without stopping the entire node or deleting the model from Gazebo.

The OFF state is visible in the state payload as:

```
enabled: false
mode: WAITING
stopped_by: DISABLED
```

9.9 Runtime shuttle addition with occupied-slot rejection

The node can spawn or control additional shuttles during simulation. The current accepted policy rejects a new shuttle if the requested start slot is occupied. This differs from the earlier experimental behavior where repeated slot use was allowed and collision avoidance handled the overlap later.

The current behavior is safer and easier to explain:

- valid slot and free space: add the shuttle;
- valid slot but occupied: reject the command;
- invalid slot: reject the command.

9.10 Simple collision avoidance added

The current collision behavior is a kinematic guard. It does not simulate physical contact. Instead, it detects that another shuttle is within 0.33 m. If movement would cross into that unsafe distance, the node reduces the step and leaves the moving shuttle waiting at the last safe pose.

This is enough for the current phase because the purpose is to prevent shuttles from visually merging and to model the expected "stop when touching another shuttle" behavior.

9.11 Room-only and full-floor compatibility

The same shuttle node works in both launch modes. The only important world parameter changes are:

- Room-only world: `-p gazebo_world_name:=room_315_only;`
- full-floor world: `-p gazebo_world_name:=mfja_3rd_floor.`

The bridge, set-pose service, and spawn service all depend on that world name. If the world name is wrong, the node can publish ROS pose topics but Gazebo will not move the visible shuttle model.

9.12 Shuttle mesh and pose alignment

The shuttle STL was updated from the corrected model file. The important practical lesson is that changing a mesh origin in CAD does not always produce the expected model-frame behavior in Gazebo if the SDF visual, collision, or link pose still applies an offset. For operational alignment, the accepted path calibration was applied to the CSV path data, and the node also keeps runtime pose calibration parameters for future tests.

The normal startup command should now run without custom scale or offset parameters.

Chapter 10

Developer Notes for Future Work

10.1 Why stoppers are not blocks yet

The current stopper implementation is a local stop primitive. It stops a shuttle at a configured point before a switch. It does not yet reserve long sections of track. Full block occupancy is a later feature. That future layer should decide whether a shuttle is allowed to enter a block based on downstream occupancy, route state, and possibly priority rules.

10.2 Why collision avoidance is not full traffic control

Collision avoidance only checks physical spacing between current shuttle poses. It is reactive. It prevents overlap but does not plan traffic. A future block system should be proactive: it should prevent a shuttle from entering a region that will become unsafe later.

10.3 Recommended next implementation phase

The recommended next phase is:

1. keep the current four start slots unless new measured entry points are accepted;
2. add named blocks to `rail_network.yaml`;
3. assign every rail segment to one or more blocks;
4. publish block occupancy in the state topic;
5. prevent shuttles from entering occupied protected blocks;
6. keep stoppers as the physical stop primitive before switches;
7. keep switch routing separate from stopper and block logic.

This preserves the clean separation already established:

```
geometry -> topology -> routing -> stopper control -> block occupancy -> Gazebo pose output
```

Chapter 11

Conclusion

The current repository is no longer just a static Gazebo map and no longer a single-package experiment. It is a ROS 2 meta-repository with dedicated description, control, bringup, and compatibility packages. The multi-robot layer provides independent namespaced control for several robot families. The Room 315 layer provides a kinematic-first shuttle system built on explicit rail topology, switch routing, stoppers, sensor-style workflow events, runtime spawning, and per-shuttle control.

The most important engineering result is separation of responsibilities: geometry is stored in package-local assets, topology is stored in `rail_network.yaml`, robot deployment is stored in YAML registries, runtime behavior is implemented in ROS 2 nodes, and the operator interface is exposed through explicit topics. This makes the project testable phase by phase, while avoiding premature contact-dynamics complexity.